

ESGI Lyon — 5ESGI ALT — Année 2025–2026

DOCUMENTATION & COMPTES RENDUS DE TP

Infrastructure & DevOps

Docker · Compose · GitHub Actions · Docker Swarm · Ansible · Traefik · Portainer · Shepherd

RÉALISÉ PAR

Souhail Cherif

ENSEIGNANT

Aurélien Leygues

APPLICATION SUPPORT

My Favorite Places (MFP)

DÉPÔT

github.com/SouhailCherif42/mfp-infra

Octobre 2026

Sommaire

1. Partie A — Documentation du projet

- a. Vue d'ensemble
- b. Travailler en local
- c. Environnements d'exécution
- d. Flows : déclencheurs et effets de bord
- e. Guide du développeur

2. Partie B — Comptes rendus des TP

- a. TP 1 — Dockerisation de MFP
- b. TP 1 bis — Intégration continue
- c. TP 2 — Docker Swarm (DinD)
- d. TP 3 — Ansible
- e. TP 4 — Traefik & Portainer
- f. TP 5 — Déploiement continu avec Shepherd

3. Bilan

Toutes les captures de ce dossier sont des captures réelles, prises sur mon poste (fenêtres Git Bash et Google Chrome) pendant la réalisation des TP. Elles sont aussi versionnées dans le dépôt, dossier `docs/captures/`, et le code présenté est celui du dépôt.

Partie A — Documentation du projet

Vue d'ensemble

My Favorite Places est composée d'une API REST Node.js / TypeScript (Express + TypeORM, authentification JWT), d'un front React (Vite) et d'une base PostgreSQL. Le projet met en place toute la chaîne qui mène d'un commit à une application déployée sur un cluster :

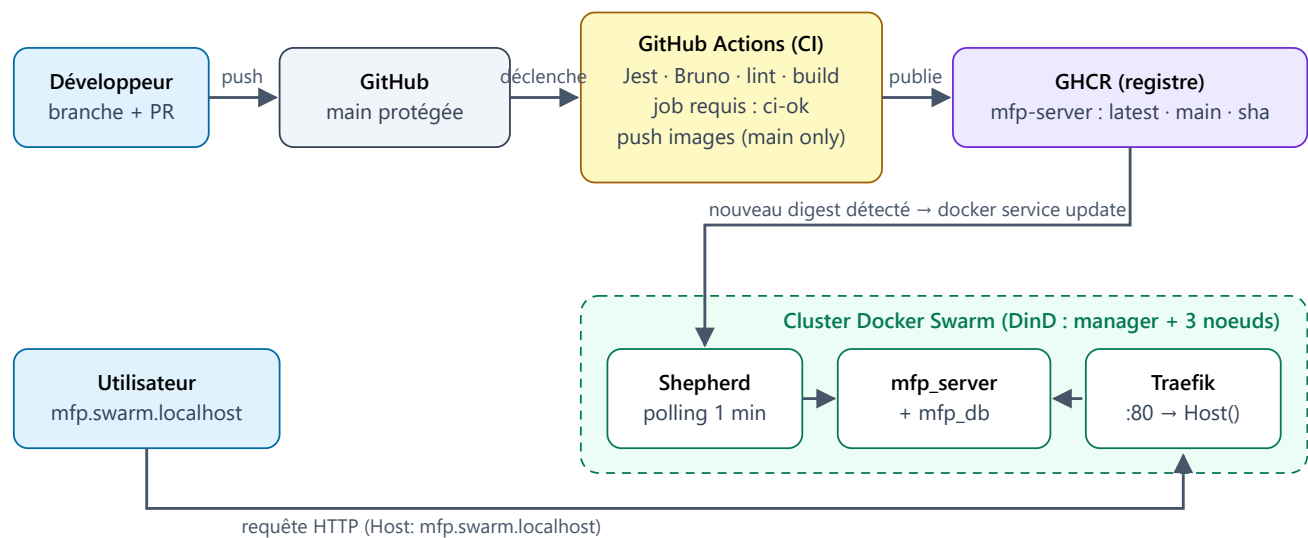


Schéma — chaîne CI/CD mise en place (du `git push` à la mise à jour du service sur Swarm).

Dossier du dépôt	Rôle
server/ , client/	Code de l'API et du front, avec leurs Dockerfiles multi-stage
compose.yml / compose.prod.yml	Stack locale construite depuis le code / la même stack avec les images publiées sur GHCR
bruno/	Collection de tests d'API (utilisée en local et dans la CI)
.github/workflows/ci.yml	Intégration continue : tests, lint, build et publication des images
swarm/ , ansible/ , ansible.sh	Cluster Swarm local (Docker-in-Docker) et son initialisation automatisée
stacks/	Stacks Swarm : Traefik, Portainer, app de vote, MFP, Shepherd

Travailler en local

Tout lancer avec Docker Compose

```
docker compose up -d --build    # db + server + client
docker compose ps               # état et healthchecks
docker compose logs -f server   # logs de l'API
docker compose down             # arrêt (le volume db-data est conservé)
```

Service	Accès	Remarque
Front (Nginx)	http://localhost:8080	Nginx relaie /api vers server:3000 → une seule origine, pas de CORS
API	http://localhost:3000/api	Démarque seulement quand la base est <i>healthy</i>
PostgreSQL	localhost:5432	Exposé pour le mode développement

Mode développement (rechargement à chaud)

```
docker compose up -d db          # uniquement la base
cd server && yarn install && yarn dev # API avec nodemon
cd client && yarn install && yarn dev  # front Vite (proxy /api → :3000)
```

Variables de l'API : `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_LOGGING` (`true` pour afficher le SQL), `SESSION_SECRET`, `PORT`.

Tester

```
cd server && yarn test          # tests unitaires Jest (getDistance)
cd bruno && bru run --env local # tests d'API Bruno (env : local, compose, swarm)
```

Reproduire la « production » (cluster Swarm) en local

```
cd swarm && docker compose up -d --build --scale node=3 --wait # 4 "machines" DinD
cd .. && ./ansible.sh                                           # init Swarm + jonction des noeuds
docker exec swarm-manager-1 docker network create --driver overlay --attachable web
# puis déploiement des stacks de stacks/ (Traefik, Portainer, MFP, Shepherd)
```

Environnements d'exécution

Environnement	Où	Images	Usage
Local / dev	Poste du développeur (Compose ou <code>yarn dev</code>)	construites depuis le code	développer, déboguer, tester avant de pousser
CI	Runners GitHub (éphémères)	construites dans le job	valider chaque PR : tests unitaires, tests d'API sur une vraie BDD, lint, build
Recette (staging)	Cluster Swarm (ici simulé en DinD)	GHCR, tag <code>main</code>	application déployée automatiquement après chaque merge, testée en conditions réelles
Production	Même architecture sur de vraies VMs (inventaire Ansible SSH)	GHCR, tag figé	clients ; passage manuel (livraison continue) plutôt qu'automatique

Dans ce projet, le cluster local joue le rôle de l'environnement de **recette** : il reçoit automatiquement chaque nouvelle version de `main` (déploiement continu). Pour la production, on préférerait un tag de version (`v1.2.0`) mis à jour volontairement.

Flows : déclencheurs et effets de bord

Déclencheur	Ce qui s'exécute	Effets de bord
Push sur une branche + Pull Request vers <code>main</code>	Workflow <code>CI</code> : détection des dossiers modifiés, Jest + build TS, test Bruno (API + BDD lancées par Compose), lint + build du front. Job final <code>ci-ok</code> .	Aucune image publiée. Statut affiché sur la PR ; merge impossible si <code>ci-ok</code> échoue.
Merge dans <code>main</code>	Même CI, puis job <code>build</code> : images <code>mfp-server</code> / <code>mfp-client</code> poussées sur GHCR (uniquement la partie modifiée).	Tags <code>latest</code> , <code>main</code> et <code>sha-xxxxxxx</code> mis à jour sur un registre public .
Lancement manuel (<i>workflow_dispatch</i>)	Tests et build des deux images, quelles que soient les modifications.	Nouvelles images publiées.
Toutes les minutes (Shepherd)	Pour chaque service portant le label <code>autoupdate_enabled=true</code> , comparaison du digest de l'image avec le registre ; <code>docker service update</code> si différent.	Redémarrage du service (<i>start-first</i> , rollback si échec), suppression de l'ancienne image sur le manager. Délai commit → recette ≈ durée de la CI + 1 min.

Effet de bord constaté : sans filtre, Shepherd interroge Docker Hub pour *tous* les services chaque minute ; le quota de pulls anonymes a été épuisé (HTTP 429), ce qui a empêché un redémarrage de PostgreSQL pendant 17 minutes (détails au TP 5). D'où le filtre par label et l'épinglage des images par digest.

Guide du développeur

- **Ne jamais pousser sur `main`** (c'est d'ailleurs refusé) : créer une branche, ouvrir une PR, attendre que `ci-ok` soit vert, merger en *squash*.
- Lancer `yarn test` et `bru run --env local` avant de pousser : la CI exécutera les mêmes commandes.
- Un merge dans `main` = **déploiement automatique en recette** dans les minutes qui suivent. Vérifier le résultat sur `http://mfp.swarm.localhost` et les logs de Shepherd dans Portainer.
- Pour qu'un nouveau service soit mis à jour automatiquement, lui ajouter le label `autoupdate_enabled=true` dans `deploy.labels` et utiliser un tag générique (`main`), jamais un tag SHA.
- Pour publier un service via Traefik : réseau `web` + labels `traefik.enable` , `traefik.http.routers.<nom>.rule=Host(...)` , `traefik.http.services.<nom>.loadbalancer.server.port` .
- **Attention** : TypeORM est en `synchronize: true` — le schéma de la base est modifié automatiquement au démarrage. Acceptable en recette, dangereux en production (prévoir des migrations).
- **Attention** : les mots de passe (BDD, secret JWT) sont en clair dans les fichiers `compose` ; en production, utiliser des *Docker secrets*.

Partie B — Comptes rendus des TP

TP 1 — Dockerisation de MFP

Le code de départ est le dépôt fourni (`esgi-2603-my-favorite-places`), copié sans son historique dans mon dépôt public.

Ex. 1.1 — Tester le serveur avec `yarn dev` et Bruno

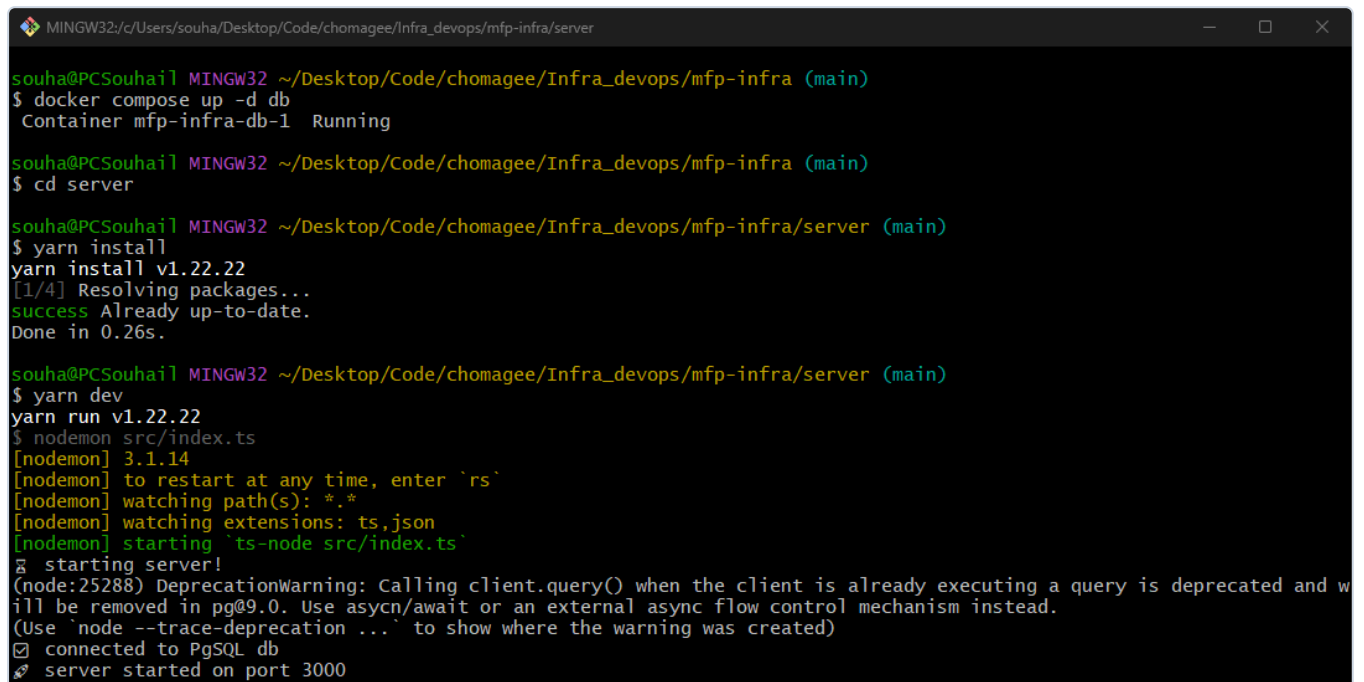
Le serveur se connectait à une base `localhost` écrite en dur. J'ai rendu la connexion configurable par variables d'environnement (indispensable pour le conteneuriser) et rendu les logs SQL optionnels :

```
server/src/datasource.ts

import { DataSource } from "typeorm";

const datasource = new DataSource({
  type: "postgres",
  host: process.env.DB_HOST || "localhost",
  port: process.env.DB_PORT ? Number(process.env.DB_PORT) : 5432,
  username: process.env.DB_USER || "postgres",
  password: process.env.DB_PASSWORD || "supersecret",
  database: process.env.DB_NAME || "postgres",
  entities: [__dirname + "/entities/**/*.js,ts"],
  logging: process.env.DB_LOGGING === "true",
  synchronize: true,
});

export default datasource;
```



```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/server
souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose up -d db
Container mfp-infra-db-1 Running

souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ cd server

souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/server (main)
$ yarn install
yarn install v1.22.22
[1/4] Resolving packages...
success Already up-to-date.
Done in 0.26s.

souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/server (main)
$ yarn dev
yarn run v1.22.22
$ nodemon src/index.ts
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/index.ts`
starting server!
(node:25288) DeprecationWarning: Calling client.query() when the client is already executing a query is deprecated and will be removed in pg@9.0. Use async/await or an external async flow control mechanism instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
connected to PostgreSQL db
server started on port 3000
```

Figure 1 — Base PostgreSQL lancée seule avec Compose, puis API en mode développement (nodemon).

Plutôt que de cliquer requête par requête, j'ai écrit une collection Bruno versionnée dans `bruno/` (création de compte, login JWT, profil, ajout et liste d'adresses, hello) exécutable en ligne de commande :

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ cd bruno

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno (main)
$ bru run --env local
1-create-user (200 OK) - 369 ms
Tests
  ✓ user created
2-login (200 OK) - 172 ms
Tests
  ✓ token returned
3-me (200 OK) - 11 ms
Tests
  ✓ authenticated user
4-add-address (200 OK) - 337 ms
Tests
  ✓ address geocoded
5-list-addresses (200 OK) - 43 ms
Tests
  ✓ one address

📄 Execution Summary


| Metric        | Result       |
|---------------|--------------|
| Status        | ✓ PASS       |
| Requests      | 5 (5 Passed) |
| Tests         | 5/5          |
| Assertions    | 0/0          |
| Duration (ms) | 2131         |



souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno (main)
$
```

Figure 2 — Collection Bruno exécutée contre l'API de développement : 5 requêtes, 5 tests OK.

Ex. 1.2 — Dockerfile du serveur

server/Dockerfile

```
# --- Stage 1 : compilation TypeScript ---
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig.json ./
COPY src src
RUN yarn build

# --- Stage 2 : image de production (JS compilé + dépendances de prod) ---
FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile --production && yarn cache clean
COPY --from=build /app/dist dist
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

L'ordre des couches suit le cours : `package.json` + `yarn install` d'abord pour profiter du cache, le code ensuite. Le premier stage compile le TypeScript, le second n'embarque que le JavaScript compilé et les dépendances de production.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/server

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ cd server

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/server (main)
$ cat Dockerfile
# --- Stage 1 : compilation TypeScript ---
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY tsconfig.json ./
COPY src src
RUN yarn build

# --- Stage 2 : image de production (JS compilé + dépendances de prod) ---
FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile --production && yarn cache clean
COPY --from=build /app/dist dist
EXPOSE 3000
CMD ["node", "dist/index.js"]

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/server (main)
$ docker build -t mfp-server . 2>&1 | tail -n 12
#14 [stage-1 5/5] COPY --from=build /app/dist dist
#14 DONE 0.0s

#15 exporting to image
#15 exporting layers 0.1s done
#15 exporting manifest sha256:e6b9f09001c5b9bfe3605df4dd5d69aa4258e90ca0a65f4a70afcfdd6dd2ab1 0.0s done
#15 exporting config sha256:14350fd6a1ee1be990305c1af19a5902ff53f4f2ea90c576170c86b1c289c5ec 0.0s done
#15 exporting attestation manifest sha256:4e7b2963195f1fc2a94fb517084f6755467040a7f3f42f6725b4a83f15b9e3dc 0.0s done
#15 exporting manifest list sha256:b07b0045662ab6f097a463af30f2ee9959505c797a4e09544025dab625f82941 0.0s done
#15 naming to docker.io/library/mfp-server:latest done
#15 unpacking to docker.io/library/mfp-server:latest 0.1s done
#15 DONE 0.4s

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/server (main)
$
```

Figure 3 — Build de l'image du serveur.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker run -d --name mfp-server --network mfp-infra_default -e DB_HOST=db -p 3000:3000 mfp-server
ff7e70b8250c709f2d7abc45d7ff84469a2639025fb4d64179c9f31062fec4ab

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ sleep 3; docker ps --filter name=mfp-server --format "table {{.Names}} \t {{.Image}} \t {{.Status}} \t {{.Ports}}
"
NAMES      IMAGE      STATUS      PORTS
mfp-server mfp-server Up 4 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker logs mfp-server
starting server!
(node:1) DeprecationWarning: Calling client.query() when the client is already executing a query is deprecated and will
be removed in pg@9.0. Use async/await or an external async flow control mechanism instead.
(Use 'node --trace-deprecation ...' to show where the warning was created)
connected to PostgreSQL db
server started on port 3000

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ cd bruno

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno (main)
$ bru run --env local | head -n 15
1-create-user (200 OK) - 152 ms
Tests
  ✓ user created
2-login (200 OK) - 81 ms
Tests
  ✓ token returned
3-me (200 OK) - 12 ms
Tests
  ✓ authenticated user
4-add-address (200 OK) - 645 ms
Tests
  ✓ address geocoded
5-list-addresses (200 OK) - 25 ms
Tests
  ✓ one address

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno (main)
$
```

Figure 4 — Conteneur lancé sur le réseau de la base, puis tests Bruno de nouveau au vert.

Ex. 1.3 et 1.4 — compose.yml, client React

Le client est construit par Vite puis servi par Nginx, qui sert aussi de passerelle vers l'API :

client/Dockerfile

```
# --- Stage 1 : build Vite (fichiers statiques dans dist/) ---
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json yarn.lock ./
RUN yarn install --frozen-lockfile
COPY . .
RUN yarn build

# --- Stage 2 : Nginx sert les fichiers statiques et proxifie /api vers le serveur ---
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

client/nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;

    # Les appels du front vers /api sont relayés au service "server" (DNS Docker)
    location /api/ {
        proxy_pass http://server:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # SPA : toutes les routes React renvoient index.html
    location / {
        try_files $uri /index.html;
    }
}
```

compose.yml

```
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_PASSWORD: supersecret
    volumes:
      - db-data:/var/lib/postgresql/data
    ports:
      - 5432:5432 # accès depuis la machine hôte pour le dev local (yarn dev)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 10
      restart: unless-stopped

  server:
    build: ./server
    environment:
      DB_HOST: db
      DB_PASSWORD: supersecret
    ports:
      - 3000:3000
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      # l'API répond 403 sans token : on vérifie seulement qu'elle répond
      test: ["CMD", "node", "-e", "fetch('http://localhost:3000/api/users/me').then(() => process.exit(0), () => process.exit(1))"]
      interval: 5s
      timeout: 3s
      retries: 10
      restart: unless-stopped

  client:
    build: ./client
    ports:
      - 8080:80
    depends_on:
      server:
        condition: service_healthy
    restart: unless-stopped

volumes:
  db-data:
```

```

MINGW32/c:/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose up -d --build 2>&1 | tail -n 14
Container mfp-infra-db-1 Creating
Container mfp-infra-db-1 Created
Container mfp-infra-server-1 Creating
Container mfp-infra-server-1 Created
Container mfp-infra-client-1 Creating
Container mfp-infra-client-1 Created
Container mfp-infra-db-1 Starting
Container mfp-infra-db-1 Started
Container mfp-infra-db-1 Waiting
Container mfp-infra-db-1 Healthy
Container mfp-infra-server-1 Starting
Container mfp-infra-server-1 Started
Container mfp-infra-client-1 Starting
Container mfp-infra-client-1 Started

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose ps --format "table {{.Name}} \t {{.Image}} \t {{.Status}} \t {{.Ports}}"
NAME                IMAGE                STATUS              PORTS
mfp-infra-client-1  mfp-infra-client    Up 1 second         0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
mfp-infra-db-1      postgres:17-alpine  Up 7 seconds (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
mfp-infra-server-1  mfp-infra-server    Up 1 second         0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp

```

Figure 5 — Stack complète : la BDD devient healthy, puis le serveur, puis le client.

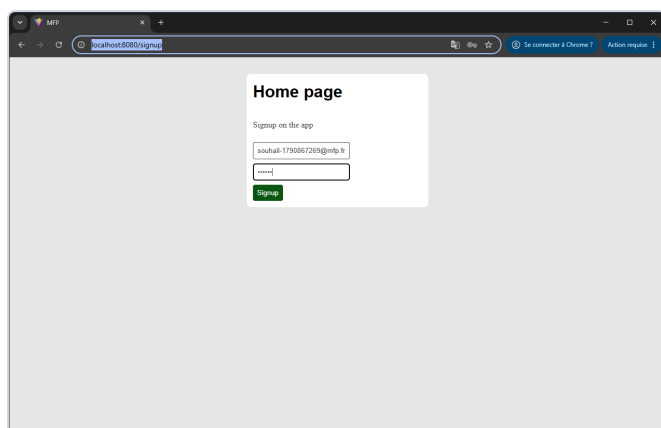


Figure 6 — Inscription sur le front servi par Nginx.

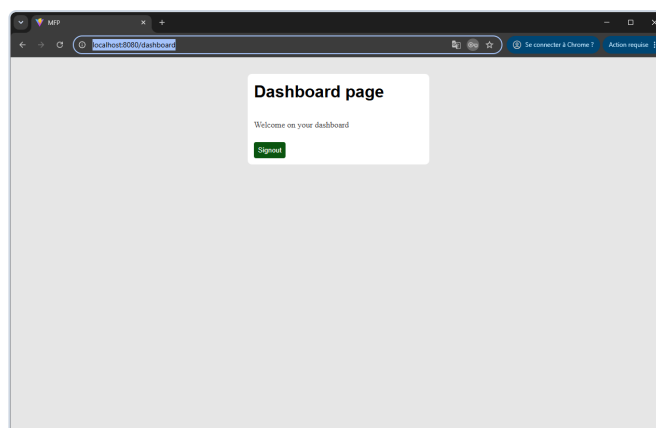


Figure 7 — Connexion réussie : tableau de bord.

Ex. 1.5 — Comment communiquent les composants ?

- **Serveur ↔ BDD** : par le réseau Docker créé par Compose ; le serveur joint la base par le **nom du service** (`DB_HOST=db`), résolu par le DNS interne de Docker. La base n'a pas besoin d'être exposée sur l'hôte.
- **Front ↔ serveur** : le front s'exécute dans le **navigateur** de l'utilisateur, pas dans le conteneur. Il appelle `/api/...` sur l'origine qui l'a servi ; Nginx relaie ces requêtes vers `server:3000`. Le navigateur ne voit qu'une seule origine (pas de CORS) et l'adresse de l'API n'est pas figée dans le build.

Pour aller plus loin

1. **Démarrer le serveur quand la BDD est prête** : healthcheck `pg_isready` sur la base + `depends_on:` `condition: service_healthy`. Même principe entre le serveur et le client.
2. **Redémarrage automatique** : `restart: unless-stopped` sur chaque service.
3. **Build stages** : l'image mono-stage pèse 473 Mo, la version multi-stage 333 Mo, soit **-140 Mo** (objectif : ~-100 Mo). Le client final (Nginx + fichiers statiques) pèse 94 Mo.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker images mfp-server
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mfp-server    multi-stage  d9d82b8b9c04   37 minutes ago  333MB
mfp-server    single-stage ed8cce2fa17d   37 minutes ago  473MB

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker images mfp-infra-client
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
mfp-infra-client latest    d2c1fc43ff25   36 minutes ago  94MB

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$
```

Figure 8 — Comparaison mesurée des tailles d'images.

TP 1 bis — Intégration continue

Ex. 2.1 et 2.2 — CI : tests puis images taguées

Un test Jest vérifie `getDistance` (distance nulle, Paris–Lyon $\approx$ 392 km, symétrie) ; il est lancé par `yarn test`.
Le workflow unique `ci.yml` enchaîne les jobs suivants :

.github/workflows/ci.yml (extrait)

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch: # lancement manuel : teste et compile tout

env:
  REGISTRY: ghcr.io
  IMAGE_PREFIX: ghcr.io/souhailcherif42/mfp

jobs:
  # Détecte quels dossiers ont changé : on ne teste / compile que ce qui est impacté
  changes:
    runs-on: ubuntu-latest
    outputs:
      server: ${ github.event_name == 'workflow_dispatch' || steps.filter.outputs.server == 'true' }
    }
      client: ${ github.event_name == 'workflow_dispatch' || steps.filter.outputs.client == 'true' }
    }
  steps:
    - uses: actions/checkout@v4
    - uses: dorny/paths-filter@v3
      id: filter
      with:
        filters: |
          server:
            - 'server/**'
            - 'bruno/**'
            - '.github/workflows/**'
          client:
            - 'client/**'
            - '.github/workflows/**'

  server-test:
    needs: changes
    if: needs.changes.outputs.server == 'true'
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: server
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: yarn
          cache-dependency-path: server/yarn.lock
      - run: yarn install --frozen-lockfile
      - run: yarn test
      - run: yarn build

  # Test d'API Bruno : l'API et sa BDD sont lancées dans la CI avec Docker Compose
  api-test:
    needs: changes
    if: needs.changes.outputs.server == 'true'
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v4
- run: docker compose up -d --build --wait db server

```

Le job `build` pousse les images sur GHCR avec `docker/metadata-action` : tags `latest`, `main` (nom de branche, utilisé par Shepherd) et `sha-xxxxxxx`. Il ne s'exécute que sur `main`, jamais sur une PR.

Le tout premier run a d'ailleurs échoué pour une vraie raison : `tsc` compilait aussi le dossier `tests/`, hors du `rootDir`. La CI a joué son rôle ; correction : `"include": ["src"]`.

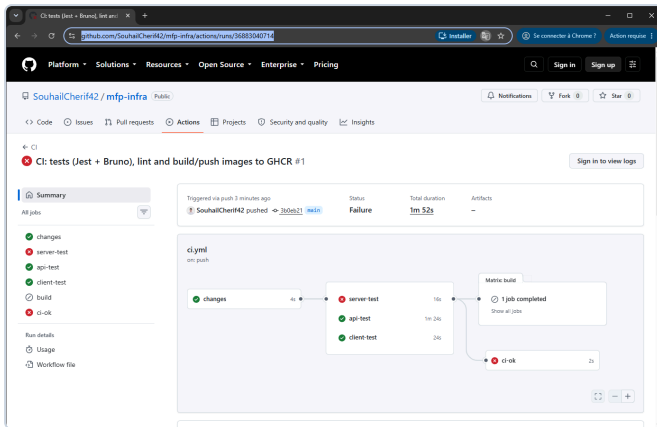


Figure 9 — Premier run : server-test en échec, ci-ok rouge, aucune image publiée.

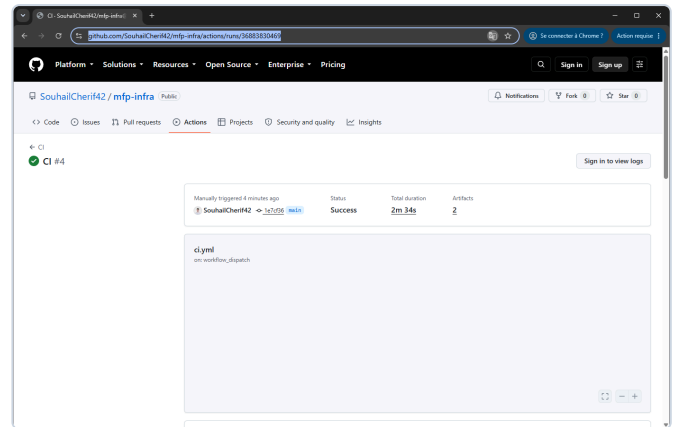


Figure 10 — Après correction : tous les jobs verts, images construites.

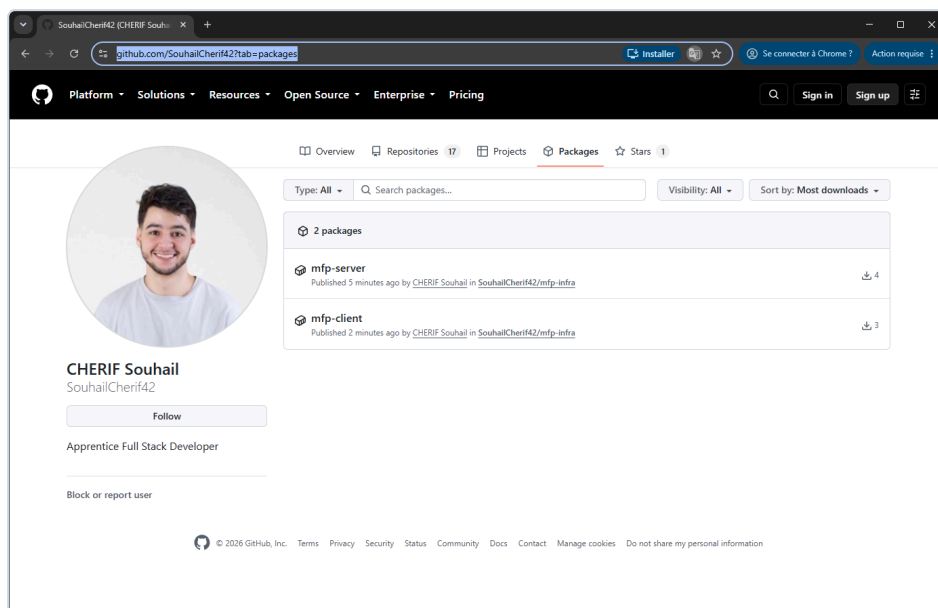


Figure 11 — Images mfp-server et mfp-client publiées sur le registre GitHub.

Ex. 2.3 — Interdire le push sur main, conditionner le merge aux tests

Protection de branche : PR obligatoire, check `ci-ok` requis, branche à jour, règles appliquées aux administrateurs. Le job `ci-ok` sert de check unique : les jobs sautés grâce au filtre de dossiers comptent comme réussis, un job en échec le fait échouer.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ git commit --allow-empty -q -m "test: push direct sur main"

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ git push
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 194 bytes | 194.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote:
remote: - Changes must be made through a pull request.
remote:
remote: - Required status check "ci-ok" is expected.
To https://github.com/SouhailCherif42/mfp-infra.git
! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to 'https://github.com/SouhailCherif42/mfp-infra.git'

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$
```

Figure 12 — Push direct sur main refusé par GitHub.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ git checkout -b test/break-getdistance
Switched to a new branch 'test/break-getdistance'

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ sed -i 's/const earthRadius = 6371/const earthRadius = 1000/' server/src/utis/getDistance.ts

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ git diff
diff --git a/server/src/utis/getDistance.ts b/server/src/utis/getDistance.ts
index 6642658..e02d5b2 100644
--- a/server/src/utis/getDistance.ts
+++ b/server/src/utis/getDistance.ts
@@ -2,7 +2,7 @@ export function getDistance(
  point1: { lng: number; lat: number },
  point2: { lng: number; lat: number },
 ): number {
-  const earthRadius = 6371; // km
+  const earthRadius = 1000; // km

  const diffLat = ((point1.lat - point2.lat) * Math.PI) / 180;
  const diffLng = ((point1.lng - point2.lng) * Math.PI) / 180;

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ git commit -qam "test: casse volontairement getDistance"

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ git push -u origin test/break-getdistance 2>&1 | tail -n 4
remote:
remote: branch 'test/break-getdistance' set up to track 'origin/test/break-getdistance'.
To https://github.com/SouhailCherif42/mfp-infra.git
 * [new branch]      test/break-getdistance -> test/break-getdistance

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ gh pr create --base main --title "Test : getDistance cassée" --body "PR de test : doit être bloquée par la CI (test Je
st en échec)."
Warning: 1 uncommitted change
Working...
Working...
https://github.com/SouhailCherif42/mfp-infra/pull/1

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$
```

Figure 13 — Branche qui casse volontairement getDistance (rayon terrestre 1000 km) et création de la PR.

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ gh pr checks 1
Working...
Some checks were not successful
0 cancelled, 2 failing, 2 successful, 2 skipped, and 0 pending checks

NAME                DESCRIPTION  ELAPSED  URL
X CI/ci-ok (pull_request)      2s       https://github.com/SouhailCherif42/mfp-infra/actions/runs/368...
X CI/server-test (pull_request) 13s       https://github.com/SouhailCherif42/mfp-infra/actions/runs/368...
- CI/build (pull_request)      -         https://github.com/SouhailCherif42/mfp-infra/actions/runs/368...
✓ CI/api-test (pull_request)    1m4s      https://github.com/SouhailCherif42/mfp-infra/actions/runs/368...
- CI/client-test (pull_request) -         https://github.com/SouhailCherif42/mfp-infra/actions/runs/368...
✓ CI/changes (pull_request)     5s        https://github.com/SouhailCherif42/mfp-infra/actions/runs/368...

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ gh run view --log-failed --job $(gh pr checks 1 --json name,link -q '[]' | select(.name=="server-test") | .link' | grep -o '[0-9]*$') | gr
ep -E "\[Expected|Received|Tests:" | cut -c 1-140
server-test Run yarn test 2026-10-01T15:29:57.3198652Z x computes Paris - Lyon distance (~392 km) (2 ms)
server-test Run yarn test 2026-10-01T15:29:57.3201393Z Expected: 392
server-test Run yarn test 2026-10-01T15:29:57.3201904Z Received: 61.45015408479944
server-test Run yarn test 2026-10-01T15:29:57.3202397Z Expected precision: -1
server-test Run yarn test 2026-10-01T15:29:57.3202876Z Expected difference: < 5
server-test Run yarn test 2026-10-01T15:29:57.3203405Z Received difference: 330.54984591520054
server-test Run yarn test 2026-10-01T15:29:57.3215464Z Tests: 1 failed, 2 passed, 3 total

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$ gh pr merge 1 --merge
Working...
X Pull request SouhailCherif42/mfp-infra#1 is not mergeable: the base branch policy prohibits the merge.
To have the pull request merged after all the requirements have been met, add the '--auto' flag.
To use administrator privileges to immediately merge the pull request, add the '--admin' flag.

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (test/break-getdistance)
$

```

Figure 14 — Le test Paris–Lyon échoue (61 km au lieu de 392) : merge refusé par la politique de branche.

Ex. 3 — compose.prod.yml

Copie de `compose.yml` où chaque `build:` est remplacé par l'image publiée (`ghcr.io/souhailcherif42/mfp-server:latest` ...).

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose -f compose.prod.yml pull 2>&1 | tail -n 4
d8b04d81c9f2 Extracting 1 s
386ced3afed0 Pull complete
d8b04d81c9f2 Pull complete
server Pulled

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose -f compose.prod.yml up -d --wait 2>&1 | tail -n 4
Container mfp-infra-client-1 Waiting
Container mfp-infra-server-1 Healthy
Container mfp-infra-db-1 Healthy
Container mfp-infra-client-1 Healthy

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose -f compose.prod.yml ps --format "table {{.Name}} {{.Image}} {{.Status}}"
NAME                IMAGE                                STATUS
mfp-infra-client-1  ghcr.io/souhailcherif42/mfp-client:latest Up 2 seconds
mfp-infra-db-1      postgres:17-alpine                 Up 13 seconds (healthy)
mfp-infra-server-1  ghcr.io/souhailcherif42/mfp-server:latest Up 7 seconds (healthy)

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ cd bruno && bru run --env compose | tail -n 16
✓ one address

Execution Summary

```

Metric	Result
Status	✓ PASS
Requests	5 (5 Passed)
Tests	5/5
Assertions	0/0
Duration (ms)	3581

```

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno (main)
$

```

Figure 15 — La stack tourne uniquement avec les images de GHCR ; les tests Bruno passent.

Pour aller plus loin

- **Compiler seulement ce qui change** : `job changes` (`dorny/paths-filter`) ; les jobs serveur ne tournent que si `server/` ou `bruno/` changent, ceux du client si `client/` change.

- **Test Bruno dans la CI** : le job `api-test` lance la BDD et l'API avec `docker compose up --wait` (healthchecks) puis exécute la collection avec la CLI Bruno.
- **Test E2E Cypress** : non réalisé.

TP 2 — Docker Swarm (cluster DinD)

Ex. 1 — DinD ou DooD ?

	Docker-in-Docker (DinD)	Docker-out-of-Docker (DooD)
Principe	Un démon Docker complet tourne dans le conteneur	Le socket de l'hôte est monté (/var/run/docker.sock)
Isolation	Images, conteneurs, réseaux propres au conteneur	Les conteneurs créés sont ceux de l'hôte
Sécurité	Exige <code>--privileged</code> : isolation rompue, quasi root sur l'hôte	Accès au socket = contrôle total du démon de l'hôte
Cas d'usage	Simuler plusieurs machines (cluster Swarm local), CI isolée	Outils qui pilotent l'hôte : Traefik, Portainer, Shepherd, Ansible en conteneur

Les deux approches donnent des droits très élevés : à réserver au local ou à des outils de confiance. Pour tester Swarm, DinD s'impose : chaque conteneur a son propre démon et peut donc devenir un nœud distinct du cluster.

Ex. 2 — Création du cluster

```
swarm/compose.yml

# Cluster Docker Swarm simulé en local : chaque conteneur DinD joue le rôle d'une machine.
# docker compose up -d --build --scale node=3 --wait → 1 manager + 3 nœuds
# puis ../ansible.sh pour initialiser le cluster Swarm
x-dind: &dind
  build: .
  privileged: true # nécessaire pour faire tourner un daemon Docker dans le conteneur (local
uniquement !)
  healthcheck:
    test: ["CMD", "docker", "info"] # le daemon interne répond
    interval: 3s
    retries: 20

services:
  manager:
    <<: *dind
    hostname: manager
    ports:
      - 80:80 # Traefik (TP4) : point d'entrée HTTP du cluster depuis la machine hôte
    volumes:
      - /home/manager # volume anonyme : les stacks copiées ici survivent à un redémarrage

node:
  <<: *dind
```

```

test: ["CMD", "docker", "info"] # le daemon interne répond
interval: 3s
retries: 20

services:
  manager:
    <<: *dind
    hostname: manager
    volumes:
      - /home/manager # volume anonyme : les stacks copiées ici survivent à un redémarrage

  node:
    <<: *dind

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker compose up -d --scale node=3 --wait
Network swarm_default Creating
Network swarm_default Created
Container swarm-manager-1 Creating
Container swarm-node-2 Creating
Container swarm-node-3 Creating
Container swarm-node-1 Creating
Container swarm-manager-1 Created
Container swarm-node-3 Created
Container swarm-node-2 Created
Container swarm-node-1 Created
Container swarm-manager-1 Starting
Container swarm-node-1 Starting
Container swarm-node-1 Started
Container swarm-manager-1 Started
Container swarm-node-2 Started
Container swarm-node-3 Started
Container swarm-node-3 Started
Container swarm-node-1 Waiting
Container swarm-node-2 Waiting
Container swarm-node-3 Waiting
Container swarm-manager-1 Waiting
Container swarm-manager-1 Healthy
Container swarm-node-1 Healthy
Container swarm-node-2 Healthy
Container swarm-node-3 Healthy

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker ps --format "table {{.Names}} {{.Image}} {{.Status}}"
NAMES          IMAGE          STATUS
swarm-node-1    docker:dind     Up 5 seconds (healthy)
swarm-node-2    docker:dind     Up 4 seconds (healthy)
swarm-node-3    docker:dind     Up 4 seconds (healthy)
swarm-manager-1 docker:dind     Up 5 seconds (healthy)

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-node-1 docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-node-1 docker version --format "Client {{.Client.Version}} / Server {{.Server.Version}}"
Client 29.8.2 / Server 29.8.2

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$

```

Figure 16 — 1 manager + 3 nœuds lancés avec --scale ; Docker est utilisable dans un nœud.

Les nœuds rejoignent le manager par le **nom du service** Compose (`manager:2377`), pas par son IP :

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 docker swarm init
Swarm initialized: current node (vutopasslubuiqi30xh62ibhx) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-5h235f187daep3nzyuonux0hvvzrnvw4jfbpd6u2cnc12dzoo9-dcaob53h1bqub57sp5apojsm 172.18.0.3:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ TOKEN=$(docker exec swarm-manager-1 docker swarm join-token -q worker)

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ for n in 1 2 3; do docker exec swarm-node-$n docker swarm join --token $TOKEN manager:2377; done
This node joined a swarm as a worker.
This node joined a swarm as a worker.
This node joined a swarm as a worker.

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 docker node ls
ID                HOSTNAME          STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
8ui66wmfo21aqwdtar77jur7s    0f3bb03117f8    Ready    Active           29.8.2
itsbkac6y1jv3lzyh1voht6    a0f79c76e493    Ready    Active           29.8.2
k08xh8b97uap33130wtz1zu1h    d67272ab7e2b    Ready    Active           29.8.2
vutopasslubuiqi30xh62ibhx *    manager          Ready    Active           Leader            29.8.2

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$

```

Figure 17 — swarm init sur le manager, jonction des 3 workers, docker node ls.

Ex. 3 — Tests du cluster

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ cat hello-world.compose.yml
services:
  hello-world:
    image: nmatsui/hello-world-api
    deploy:
      replicas: 2

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 mkdir -p /home/manager

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker cp hello-world.compose.yml swarm-manager-1:/home/manager/

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec -w /home/manager swarm-manager-1 docker stack deploy -c hello-world.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello-world

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$
```

Figure 18 — Stack hello-world copiée dans /home/manager (volume anonyme) puis déployée.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
4825rjv488kk     hello_hello-world    replicated          2/2                 nmatsui/hello-world-api:latest

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 docker service ps hello_hello-world --format "table {{.Name}}\t{{.Node}}\t{{.CurrentState}}\n"
NAME                NODE                CURRENT STATE
hello_hello-world.1  manager            Running 9 seconds ago
hello_hello-world.2  d67272ab7e2b       Running 9 seconds ago

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ for c in swarm-manager-1 swarm-node-1 swarm-node-2 swarm-node-3; do echo "== $c"; docker exec $c docker ps --format "{{.Names}}\t{{.Status}}"; done
== swarm-manager-1
hello_hello-world.1.e1wwa85oiwja7v1uuoe4dfj6n  Up 11 seconds
== swarm-node-1
hello_hello-world.2.1t5a0of07qga39r7ii426o3wl  Up 12 seconds
== swarm-node-2
== swarm-node-3

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$
```

Figure 19 — 2 répliques : un sur le manager, un sur un worker ; les autres nœuds n'ont rien.

Déduction : on ne choisit pas la machine, on décrit un état souhaité (2 répliques) et l'ordonnanceur Swarm répartit les tâches sur les nœuds disponibles, sur le réseau overlay commun. Les contraintes et les modes changent ce placement :

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ cat hello-world.manager.compose.yml
services:
  hello-world:
    image: nmatsui/hello-world-api
    deploy:
      replicas: 2
      placement:
        constraints: [node.role == manager] # uniquement sur le(s) manager(s)

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec -w /home/manager swarm-manager-1 docker stack deploy -c hello-world.manager.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service hello_hello-world (id: 4825rjv488kk074ym3aoq4h73)

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ sleep 15; docker exec swarm-manager-1 docker service ps hello_hello-world --filter desired-state=running --format "table {{.Name}}\t{{.Node}}\t{{.CurrentState}}\n"
NAME                NODE                CURRENT STATE
hello_hello-world.1  manager            Running 53 seconds ago
hello_hello-world.2  manager            Running 11 seconds ago

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$
```

Figure 20 — placement `node.role == manager` : les 2 répliques sur le manager.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ cat hello-world.global.compose.yml
services:
  hello-world:
    image: nmatsui/hello-world-api
    deploy:
      mode: global # exactement 1 conteneur sur chaque nœud du cluster

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 docker stack rm hello
Removing service hello_hello-world
Removing network hello_default

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ sleep 10; docker exec -w /home/manager swarm-manager-1 docker stack deploy -c hello-world.global.compose.yml hello
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello_default
Creating service hello_hello-world

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ sleep 20; docker exec swarm-manager-1 docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
jpe1cczw4sve     hello_hello-world    global              4/4                 nmatsui/hello-world-api:latest

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$ docker exec swarm-manager-1 docker service ps hello_hello-world --format "table {{.Name}}\t{{.Node}}\t{{.CurrentState}}\n"
NAME                NODE                CURRENT STATE
hello_hello-world.8u166mfo2laqdtar77jur7s     0f3bb0311f78       Running 9 seconds ago
hello_hello-world.it3kxady1y3l2zyhvh0t6       adf9c7ee93         Running 9 seconds ago
hello_hello-world.k08xh8b9uap33130wtz1zuth     d67272ab7e2b       Running 20 seconds ago
hello_hello-world.vutopass1ubu1q130xh621btx     manager            Running 20 seconds ago

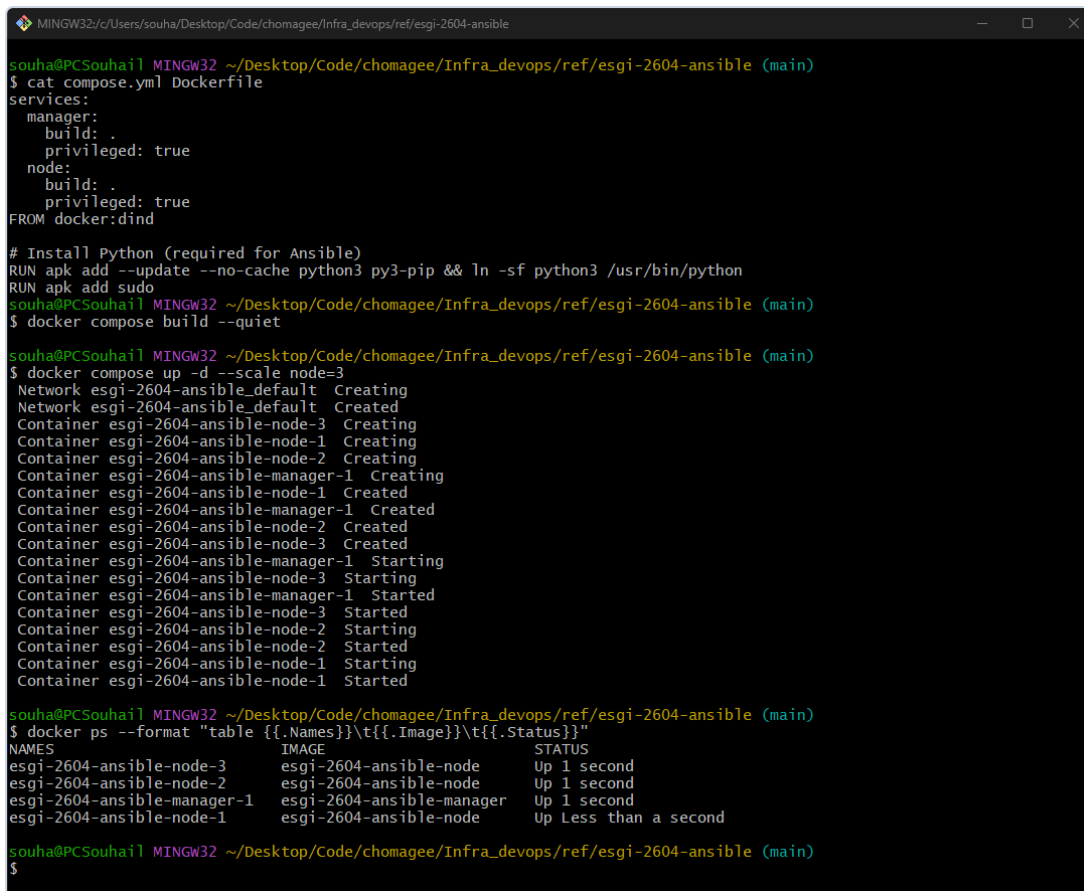
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/swarm (tp2-swarm)
$
```

Figure 21 — mode : global : exactement une tâche par nœud (4/4).

TP 3 — Automatisation avec Ansible

Ex. 4.1 — 3 nœuds sans modifier compose.yml

Option `--scale` de Compose : `docker compose up -d --scale node=3`.



```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ cat compose.yml Dockerfile
services:
  manager:
    build: .
    privileged: true
  node:
    build: .
    privileged: true
FROM docker:dind

# Install Python (required for Ansible)
RUN apk add --update --no-cache python3 py3-pip && ln -sf python3 /usr/bin/python
RUN apk add sudo
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker compose build --quiet

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker compose up -d --scale node=3
Network esgi-2604-ansible_default Creating
Network esgi-2604-ansible_default Created
Container esgi-2604-ansible-node-3 Creating
Container esgi-2604-ansible-node-1 Creating
Container esgi-2604-ansible-node-2 Creating
Container esgi-2604-ansible-manager-1 Creating
Container esgi-2604-ansible-node-1 Created
Container esgi-2604-ansible-manager-1 Created
Container esgi-2604-ansible-node-2 Created
Container esgi-2604-ansible-node-3 Created
Container esgi-2604-ansible-manager-1 Starting
Container esgi-2604-ansible-node-3 Starting
Container esgi-2604-ansible-manager-1 Started
Container esgi-2604-ansible-node-3 Started
Container esgi-2604-ansible-node-2 Starting
Container esgi-2604-ansible-node-2 Started
Container esgi-2604-ansible-node-1 Starting
Container esgi-2604-ansible-node-1 Started

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"
NAMES          IMAGE              STATUS
esgi-2604-ansible-node-3   esgi-2604-ansible-node   Up 1 second
esgi-2604-ansible-node-2   esgi-2604-ansible-node   Up 1 second
esgi-2604-ansible-manager-1 esgi-2604-ansible-manager Up 1 second
esgi-2604-ansible-node-1   esgi-2604-ansible-node   Up Less than a second

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$
```

Figure 22 — Dépôt fourni : image DinD + Python (requis par Ansible), 4 conteneurs lancés.

Ex. 4.2 — Le playbook fourni, exécuté depuis Windows

Le playbook initialise Swarm sur le manager, récupère le jeton worker et fait rejoindre les workers. Ansible ne fonctionne pas nativement sous Windows : je l'exécute dans un conteneur (Python + ansible-core + community.docker), avec le socket Docker monté (DooD). La connexion `community.docker.docker` remplace SSH par des `docker exec`.

```
ansible/Dockerfile

# Ansible n'est pas supporté nativement sous Windows : on l'exécute dans un conteneur.
# Le socket Docker de l'hôte est monté (DooD) pour piloter les conteneurs DinD du cluster
# via la connexion community.docker.docker (équivalent d'un "docker exec").
FROM python:3.12-alpine
RUN apk add --no-cache docker-cli \
    && pip install --no-cache-dir ansible-core \
    && ansible-galaxy collection install community.docker
WORKDIR /work
```

Première exécution : **échec réel** des workers. Le playbook fourni les fait rejoindre `router:2377`, un hôte qui n'existe pas (*lookup router ... server misbehaving*).

```
MINGW32/c:/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible
TASK [Set worker join token as a fact] *****
ok: [esgi-2604-ansible-manager-1]

TASK [Check join command (for workers)] *****
ok: [esgi-2604-ansible-manager-1] => {
  "worker_join_command": "docker swarm join --token SWMTKN-1-613o9lii66xewfwmz13dawezzxves6s3cgytr1jwa1ako1j28-by9nx4sfo4quupxahvtpw4k40"
}

PLAY [Join workers to the Swarm cluster] *****

TASK [Gathering Facts] *****
[WARNING]: Host 'esgi-2604-ansible-node-3' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of a
nother Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_app
endices/interpreter_discovery.html for more information.
[WARNING]: Host 'esgi-2604-ansible-node-2' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of a
nother Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_app
endices/interpreter_discovery.html for more information.
[WARNING]: Host 'esgi-2604-ansible-node-1' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of a
nother Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.21/reference_app
endices/interpreter_discovery.html for more information.
ok: [esgi-2604-ansible-node-3]
ok: [esgi-2604-ansible-node-2]
ok: [esgi-2604-ansible-node-1]

TASK [Join swarm as worker] *****
[ERROR]: Task failed: Module failed: A 'failed_when' expression evaluated to 'True'.
Origin: /work/ansible/init_swarm_cluster.yml:28:7

26 become: yes
27 tasks:
28   - name: Join swarm as worker
    ^ column 7

fatal: [esgi-2604-ansible-node-3]: FAILED! => [{"changed": true, "cmd": ["docker", "swarm", "join", "--token", "SWMTKN-1-613o9lii66xewfwmz13dawezzxves6s3cgytr1jwa1ako1j28-by9nx4sfo4quupxahvtpw4k40", "router:2377"], "delta": "0:00:08.040404", "end": "2026-10-01 15:44:32.032304", "failed_when_result": true, "failed_when_suppressed_exception": "(traceback unavailable)", "msg": "The command exited with a non-zero return code.", "rc": 1, "start": "2026-10-01 15:44:23.991900", "stderr": "Error response from daemon: could not find local IP address: dial udp: lo okup router on 127.0.0.11:53: server misbehaving", "stdout_lines": ["Error response from daemon: could not find local IP address: dial udp: lookup router on 127.0.0.11:53: server misbehaving"], "stdout": "", "stdout_lines": []}]
fatal: [esgi-2604-ansible-node-1]: FAILED! => [{"changed": true, "cmd": ["docker", "swarm", "join", "--token", "SWMTKN-1-613o9lii66xewfwmz13dawezzxves6s3cgytr1jwa1ako1j28-by9nx4sfo4quupxahvtpw4k40", "router:2377"], "delta": "0:00:08.040108", "end": "2026-10-01 15:44:32.032504", "failed_when_result": true, "failed_when_suppressed_exception": "(traceback unavailable)", "msg": "The command exited with a non-zero return code.", "rc": 1, "start": "2026-10-01 15:44:23.992396", "stderr": "Error response from daemon: could not find local IP address: dial udp: lo okup router on 127.0.0.11:53: server misbehaving", "stdout_lines": ["Error response from daemon: could not find local IP address: dial udp: lookup router on 127.0.0.11:53: server misbehaving"], "stdout": "", "stdout_lines": []}]
fatal: [esgi-2604-ansible-node-2]: FAILED! => [{"changed": true, "cmd": ["docker", "swarm", "join", "--token", "SWMTKN-1-613o9lii66xewfwmz13dawezzxves6s3cgytr1jwa1ako1j28-by9nx4sfo4quupxahvtpw4k40", "router:2377"], "delta": "0:00:08.046822", "end": "2026-10-01 15:44:32.054327", "failed_when_result": true, "failed_when_suppressed_exception": "(traceback unavailable)", "msg": "The command exited with a non-zero return code.", "rc": 1, "start": "2026-10-01 15:44:24.007505", "stderr": "Error response from daemon: could not find local IP address: dial udp: lo okup router on 127.0.0.11:53: server misbehaving", "stdout_lines": ["Error response from daemon: could not find local IP address: dial udp: lookup router on 127.0.0.11:53: server misbehaving"], "stdout": "", "stdout_lines": []}]

PLAY RECAP *****
esgi-2604-ansible-manager-1 : ok=5 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
esgi-2604-ansible-node-1 : ok=1 changed=0 unreachable=0 failed=1 skipped=0 rescued=0 ignored=0
esgi-2604-ansible-node-2 : ok=1 changed=0 unreachable=0 failed=1 skipped=0 rescued=0 ignored=0
esgi-2604-ansible-node-3 : ok=1 changed=0 unreachable=0 failed=1 skipped=0 rescued=0 ignored=0

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$
```

Figure 23 — Erreur de jonction des workers : l'hôte « router » est introuvable.

```
MINGW32/c:/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ sed -i 's/ router:2377/ manager:2377/' ansible/init_swarm_cluster.yml

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ sed -i 's/ansible_connection=community.docker.docker/docker\ansible_connection=community.docker.docker\nansible_python_interpreter=/usr/bin/python3/' ansible/inventory.ini

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ git diff
warning: in the working copy of 'ansible/init_swarm_cluster.yml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'ansible/inventory.ini', LF will be replaced by CRLF the next time Git touches it
diff --git a/ansible/init_swarm_cluster.yml b/ansible/init_swarm_cluster.yml
index 57bc155..9b0b67f 100644
--- a/ansible/init_swarm_cluster.yml
+++ b/ansible/init_swarm_cluster.yml
@@ -26,6 +26,6 @@
 become: yes
 tasks:
   - name: Join swarm as worker
     command: "{{ hostvars[groups['managers'][0]]['worker_join_command'] }}" router:2377"
+    command: "{{ hostvars[groups['managers'][0]]['worker_join_command'] }}" manager:2377"
     register: command_result
     failed_when: "This node is already part of a swarm" not in command_result.stderr and command_result.rc != 0"
diff --git a/ansible/inventory.ini b/ansible/inventory.ini
index 1374c36..a2df8ed 100644
--- a/ansible/inventory.ini
+++ b/ansible/inventory.ini
@@ -4,6 +4,7 @@
esgi-2604-ansible-manager-1

[managers:vars]
ansible_connection=community.docker.docker
+ansible_python_interpreter=/usr/bin/python3

[workers]
# it should be the docker container name of the worker nodes, in our case "node" prefixed by docker compose
@@ -12,4 +13,5 @@
esgi-2604-ansible-node-2
esgi-2604-ansible-node-3

[workers:vars]
-ansible_connection=community.docker.docker
\ No newline at end of file
+ansible_connection=community.docker.docker
+ansible_python_interpreter=/usr/bin/python3
\ No newline at end of file

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$
```

Figure 24 — Correction : manager:2377 + interpréteur Python explicite dans l'inventaire.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker run --rm -e ANSIBLE_FORCE_COLOR=1 -v /var/run/docker.sock:/var/run/docker.sock -v "$(pwd -W)":/work mfp-ansible ansible-playbook -i
ansible/inventory.ini ansible/init_swarm_cluster.yml | tail -n 22
ok: [esgi-2604-ansible-manager-1] => {
  "worker_join_command": "docker swarm join --token SWMTKN-1-613o9lii66xewfwmzil3dawezzxves6s3cqytr1jwa1ako1j28-by9nx4sfo4quupxahvtpw4k40"
}

PLAY [Join workers to the Swarm cluster] *****

TASK [Gathering Facts] *****
ok: [esgi-2604-ansible-node-2]
ok: [esgi-2604-ansible-node-3]
ok: [esgi-2604-ansible-node-1]

TASK [Join swarm as worker] *****
changed: [esgi-2604-ansible-node-3]
changed: [esgi-2604-ansible-node-2]
changed: [esgi-2604-ansible-node-1]

PLAY RECAP *****
esgi-2604-ansible-manager-1 : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-1   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-2   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-3   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker exec esgi-2604-ansible-manager-1 docker node ls
ID                HOSTNAME                STATUS      AVAILABILITY    MANAGER STATUS  ENGINE VERSION
kahtqesw1288hd9wihkdcwmw9    3af138f65f32    Ready      Active           Ready           29.8.2
mr0bn1xytfxburogwmws1468n    9dd831d176c0    Ready      Active           Ready           29.8.2
07912qg578td8tyjx7n9hngc    19b12476aa7c    Ready      Active           Leader          29.8.2
vs5b137r4xkxoiwd1c9l9p1c    b3c404e3e3f2    Ready      Active           Ready           29.8.2

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$
```

Figure 25 — Après correction : manager initialisé, 3 workers joints, cluster opérationnel.

Ex. 4.3 — Relancer le même playbook

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker run --rm -e ANSIBLE_FORCE_COLOR=1 -v /var/run/docker.sock:/var/run/docker.sock -v "$(pwd -W)":/work mfp-ansible ansible-playbook -i
ansible/inventory.ini ansible/init_swarm_cluster.yml | tail -n 14
ok: [esgi-2604-ansible-node-2]
ok: [esgi-2604-ansible-node-3]

TASK [Join swarm as worker] *****
changed: [esgi-2604-ansible-node-2]
changed: [esgi-2604-ansible-node-1]
changed: [esgi-2604-ansible-node-3]

PLAY RECAP *****
esgi-2604-ansible-manager-1 : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-1   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-2   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-3   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker exec esgi-2604-ansible-node-1 docker swarm join --token xxx manager:2377
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join another one.

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker exec esgi-2604-ansible-manager-1 docker node ls --format "table {{.Hostname}}\t{{.Status}}\t{{.ManagerStatus}}"
HOSTNAME                STATUS      MANAGER STATUS
3af138f65f32    Ready
9dd831d176c0    Ready
19b12476aa7c    Ready      Leader
b3c404e3e3f2    Ready

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$
```

Figure 26 — Relance : aucune erreur, cluster inchangé... mais Ansible annonce « changed ».

Constat : le résultat est stable (les erreurs « already part of a swarm » sont tolérées par `failed_when`), mais le module `command` rapporte toujours `changed`. Ansible ne sait pas si la commande a réellement modifié quelque chose : ce n'est pas une vraie idempotence.

Ex. 5.1 — Ajouter un nœud

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker compose up -d --scale node=4 2>&1 | grep node-4
Container esgi-2604-ansible-node-4 Creating
Container esgi-2604-ansible-node-4 Created
Container esgi-2604-ansible-node-4 Starting
Container esgi-2604-ansible-node-4 Started

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ sed -i 's/esgi-2604-ansible-node-3/esgi-2604-ansible-node-4/' ansible/inventory.ini

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ cat ansible/inventory.ini
[managers]
# it should be the docker container name of the manager node, in our case "manager" prefixed by docker compose
esgi-2604-ansible-manager-1

[managers:vars]
ansible_connection=community.docker.docker
ansible_python_interpreter=/usr/bin/python3

[workers]
# it should be the docker container name of the worker nodes, in our case "node" prefixed by docker compose
esgi-2604-ansible-node-1
esgi-2604-ansible-node-2
esgi-2604-ansible-node-3
esgi-2604-ansible-node-4

[workers:vars]
ansible_connection=community.docker.docker
ansible_python_interpreter=/usr/bin/python3
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ sleep 5; docker run --rm -e ANSIBLE_FORCE_COLOR=1 -v /var/run/docker.sock:/var/run/docker.sock -v "$(pwd -w)":/work mfp-ansible ansible-playbook -i ansible/inventory.ini ansible/init_swarm_cluster.yml | tail -n 7
PLAY RECAP *****
esgi-2604-ansible-manager-1 : ok=5    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-1   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-2   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-3   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
esgi-2604-ansible-node-4   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$ docker exec esgi-2604-ansible-manager-1 docker node ls --format "table {{.Hostname}}\t{{.Status}}\t{{.ManagerStatus}}"
HOSTNAME      STATUS      MANAGER STATUS
3af138f65f32  Ready
9dd831d176c0  Ready
19b12476aa7c  Ready      Leader
b3c404e3e3f2  Ready
cf24aeb9557d  Ready

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/esgi-2604-ansible (main)
$
```

Figure 27 — --scale node=4, ajout de node-4 dans l'inventaire, relance : 5 nœuds.

Ex. 5.2 — Sur de vraies VMs / VPS

- **Inventaire** : IP des machines (`ansible_host`), `ansible_connection=ssh` , `ansible_user` , clé SSH, `ansible_become=true` (sudo) ; adresse IP privée du manager pour la jonction.
- **Playbook** : ajouter l'installation de Docker (dépôt apt officiel ou rôle `geerlingguy.docker`), `swarm init --advertise-addr <IP privée>` , et ouvrir les ports Swarm 2377/tcp, 7946/tcp+udp, 4789/udp.

Le bonus VirtualBox n'a pas été réalisé.

Ex. 5.3 — Ansible dans mon dépôt

Mon dossier `swarm/` fournit l'image DinD + Python ; `ansible.sh` construit le conteneur Ansible et lance le playbook (fonctionne sous Git Bash comme sous Linux). Le playbook est réécrit pour être **réellement idempotent** : il lit l'état Swarm de chaque nœud et n'agit que si nécessaire.

ansible/init_swarm_cluster.yml

```
# Initialise un cluster Docker Swarm : idempotent, rejouable sans effet si tout est déjà en place.
- name: Initialize Docker Swarm manager
  hosts: managers
  tasks:
    - name: Read swarm state
      command: docker info --format {% raw %}'{{.Swarm.LocalNodeState}}' {% endraw %}
      register: swarm_state
      changed_when: false

    - name: Initialize swarm
      command: docker swarm init
      when: swarm_state.stdout != "active"

    - name: Retrieve worker join token
      command: docker swarm join-token worker -q
      register: worker_token
      changed_when: false

- name: Join workers to the Swarm cluster
  hosts: workers
  vars:
    manager: "{{ groups['managers'][0] }}"
  tasks:
    - name: Read swarm state
      command: docker info --format {% raw %}'{{.Swarm.LocalNodeState}}' {% endraw %}
      register: swarm_state
      changed_when: false

    - name: Join swarm as worker
      command: >-
        docker swarm join --token {{ hostvars[manager].worker_token.stdout }}
        {{ swarm_manager_addr }}:2377
      when: swarm_state.stdout != "active"
```

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp3-ansible)
$ ./ansible.sh | tail -n 30

TASK [Initialize swarm] *****
changed: [swarm-manager-1]

TASK [Retrieve worker join token] *****
ok: [swarm-manager-1]

PLAY [Join workers to the Swarm cluster] *****

TASK [Gathering Facts] *****
ok: [swarm-node-3]
ok: [swarm-node-2]
ok: [swarm-node-1]

TASK [Read swarm state] *****
ok: [swarm-node-1]
ok: [swarm-node-2]
ok: [swarm-node-3]

TASK [Join swarm as worker] *****
changed: [swarm-node-2]
changed: [swarm-node-1]
changed: [swarm-node-3]

PLAY RECAP *****
swarm-manager-1      : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
swarm-node-1         : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
swarm-node-2         : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
swarm-node-3         : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp3-ansible)
$ docker exec swarm-manager-1 docker node ls
ID                                HOSTNAME        STATUS      AVAILABILITY    MANAGER STATUS  ENGINE VERSION
uj91yum5eottv3w43k6x53bit         5b5bc38177e3    Ready      Active           Ready           29.8.2
v24d514ru3a4yub8mirh9m6vb        a47b250bb0e1    Ready      Active           Ready           29.8.2
99c4xwule1xhgoiiv1vrkx61x        c9a838dc7311    Ready      Active           Ready           29.8.2
notreqyuehqohenj9akqft6um *      manager         Ready      Active           Leader          29.8.2

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp3-ansible)
$

```

Figure 28 — Premier passage : init + 3 jonctions (changed=1 par machine).

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp3-ansible)
$ ./ansible.sh | tail -n 8
skipping: [swarm-node-3]

PLAY RECAP *****
swarm-manager-1      : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
swarm-node-1         : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
swarm-node-2         : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
swarm-node-3         : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp3-ansible)
$

```

Figure 29 — Deuxième passage : changed=0, les actions sont sautées.

Ex. 5.4 — Ansible et Terraform

Terraform crée l'infrastructure de façon déclarative (VMs, réseaux, DNS, load balancers chez un hébergeur), en gardant un état (*state*) pour calculer les différences. **Ansible** configure ensuite l'intérieur des machines (paquets, Docker, Swarm). Usage combiné : Terraform provisionne les VPS et fournit leurs IP, qui alimentent l'inventaire Ansible.

TP 4 — Traefik & Portainer

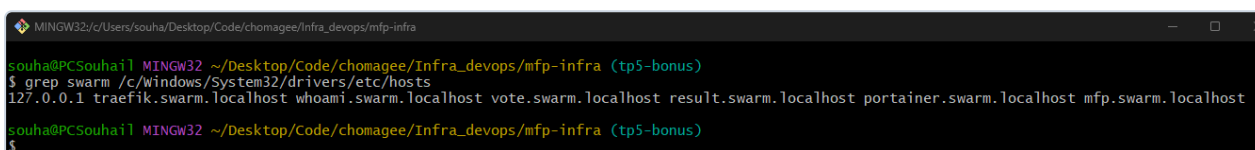
Un **reverse proxy** (couche 7, HTTP) reçoit toutes les requêtes sur les ports 80/443 et choisit l'application cible d'après l'en-tête `Host`, ce qui évite d'exposer un port par application. Traefik lit sa configuration directement dans les **labels** des services Swarm.

Ex. 1 — Déployer Traefik

Le manager publie le port 80 sur l'hôte (`swarm/compose.yml`) ; le réseau `web` est un **overlay** pour que Traefik, sur le manager, atteigne des conteneurs placés sur d'autres nœuds.

stacks/traefik.yml (extrait)

```
services:
  traefik:
    image: traefik:v3.6
    networks:
      # Connect to the 'web' overlay network for inter-container communication across nodes
      - web
    ports:
      - 80:80 # HTTP
      # - 443:443 # HTTPS - Would work only on a server
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro # Swarm API socket
    command:
      # HTTP EntryPoint
      - "--entrypoints.web.address=:80"
      # Enable the Docker Swarm provider (instead of Docker provider)
      - "--providers.swarm.endpoint=unix:///var/run/docker.sock"
      # Watch for Swarm service changes (requires socket access)
      - "--providers.swarm.watch=true"
      # Recommended: Don't expose services by default; require explicit labels
      - "--providers.swarm.exposedbydefault=false"
      # Specify the default network for Traefik to connect to services
      - "--providers.swarm.network=web"
      # API & Dashboard
      - "--api.dashboard=true" # Enable the dashboard
      - "--api.insecure=true" # Explicitly disable insecure API mod
      # Observability
      - "--log.level=INFO" # Set the Log Level e.g INFO, DEBUG
      - "--accesslog=true" # Enable Access Logs
      # - "--metrics.prometheus=true" # Enable Prometheus
    deploy:
      mode: replicated
      replicas: 1
      placement:
        # Placement constraints restrict where Traefik tasks can run.
        # Running on manager nodes is common for accessing the Swarm API via the socket.
        constraints:
          - node.role == manager
      # Traefik Dynamic configuration via labels
      # In Swarm, labels on the service definition configure Traefik routing for that service.
      labels:
        - "traefik.enable=true"
```



```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouha11 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ grep swarm /c/windows/System32/drivers/etc/hosts
127.0.0.1 traefik.swarm.localhost whoami.swarm.localhost vote.swarm.localhost result.swarm.localhost portainer.swarm.localhost mfp.swarm.localhost
souha@PCSouha11 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$
```

Figure 30 — Fichier hosts : les noms *.swarm.localhost pointent vers la machine locale.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 docker service ls
ID                NAME                MODE                REPLICAS            IMAGE                PORTS
tlugk085bgf0     traefik_traefik     replicated          1/1                 traefik:v3.6        *:80->80/tcp
qqedw0rmwp7z     traefik_whoami      replicated          1/1                 traefik/whoami:latest

souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 docker service ps traefik_whoami --format "table {{.Name}}\t{{.Node}}\t{{.CurrentState}}"
NAME                NODE                CURRENT STATE
traefik_whoami.1    5b5bc38177e3       Running 27 seconds ago

souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ curl -s http://whoami.swarm.localhost
Hostname: 1cc740811512
IP: 127.0.0.1
IP: ::1
IP: 10.0.1.3
IP: 172.19.0.3
RemoteAddr: 10.0.1.6:56350
GET / HTTP/1.1
Host: whoami.swarm.localhost
User-Agent: curl/8.7.1
Accept: */*
Accept-Encoding: gzip
X-Forwarded-For: 10.0.0.2
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Port: 80
X-Forwarded-Proto: http
X-Forwarded-Server: ee5c998a08e8
X-Real-Ip: 10.0.0.2

souha@PCSouhai1 MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$
```

Figure 31 — whoami tourne sur un worker et répond via Traefik (X-Forwarded-*).

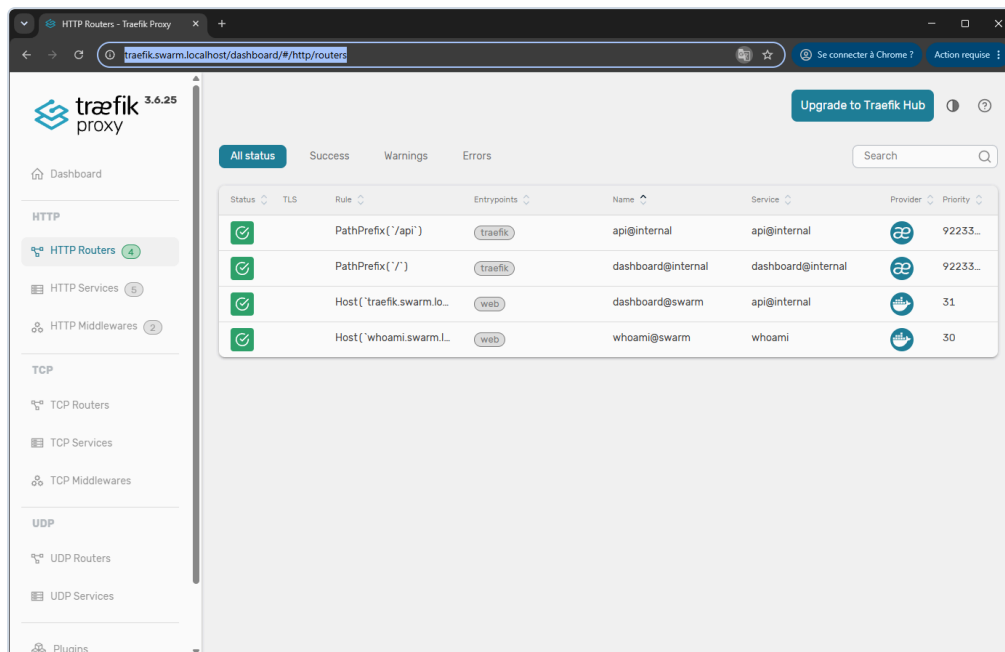


Figure 32 — Dashboard Traefik : routeurs déclarés automatiquement depuis les labels.

Ex. 2 — L'application de vote

Fonctionnement : *vote* (Python) pousse chaque vote dans *Redis* ; le *worker* (.NET) les dépile et les enregistre dans *PostgreSQL* ; *result* (Node.js) lit la base et affiche le résultat en temps réel. Les réseaux `frontend` et `backend` isolent les deux moitiés, seul le worker est sur les deux.

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/ref/example-voting-app
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/example-voting-app (main)
$ docker compose -f docker-compose.images.yml up -d --wait 2>&1 | tail -n 6
Container example-voting-app-worker-1 Waiting
Container example-voting-app-db-1 Healthy
Container example-voting-app-result-1 Healthy
Container example-voting-app-redis-1 Healthy
Container example-voting-app-worker-1 Healthy
Container example-voting-app-vote-1 Healthy

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/example-voting-app (main)
$ docker compose -f docker-compose.images.yml ps --format "table {{.Service}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
SERVICE IMAGE STATUS PORTS
db postgres:15-alpine Up 7 seconds (healthy) 5432/tcp
redis redis:alpine Up 7 seconds (healthy) 6379/tcp
result dockersamples/examplevotingapp_result Up 1 second 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
vote dockersamples/examplevotingapp_vote Up 1 second 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
worker dockersamples/examplevotingapp_worker Up 1 second
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/ref/example-voting-app (main)
$

```

Figure 33 — Test local avec Docker Compose.

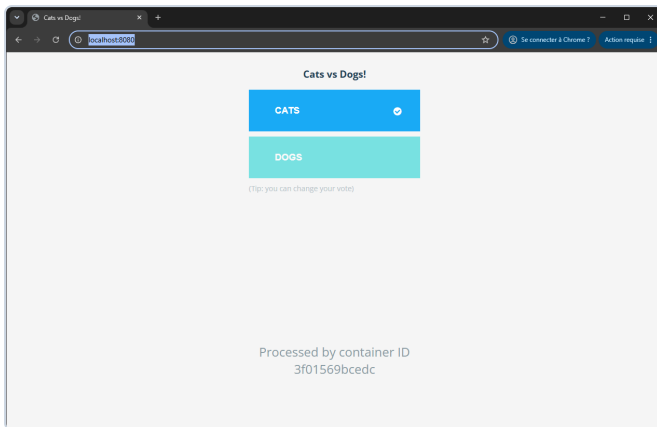


Figure 34 — Vote en local (port 8080).

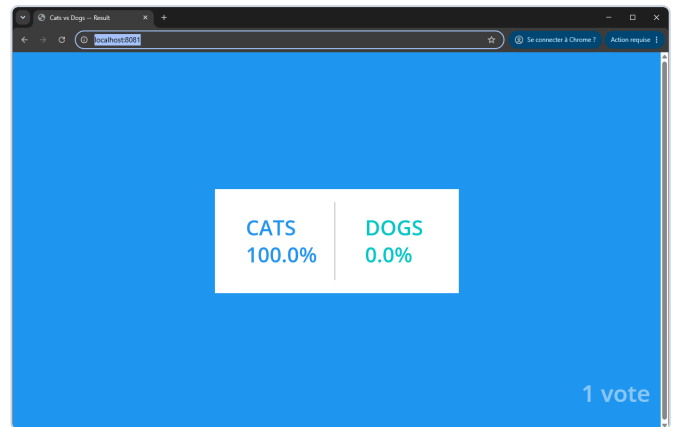


Figure 35 — Résultat en local (port 8081).

```

MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 docker stack services voting
ID NAME MODE REPLICAS IMAGE PORTS
do9517lso4uf voting_db replicated 1/1 postgres:15-alpine
nniflpk70jtt voting_redis replicated 1/1 redis:alpine
s2ina04v28t4 voting_result replicated 1/1 dockersamples/examplevotingapp_result:latest *:8081->80/tcp
ydyex57lyu9o voting_vote replicated 2/2 dockersamples/examplevotingapp_vote:latest *:8080->80/tcp
fchpruwvd08k voting_worker replicated 2/2 dockersamples/examplevotingapp_worker:latest

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 sh -c "wget -qO- http://localhost:8080 | grep -o '<title>.*</title>'; wget -qO- http://localhost:8081 | grep -o '<title>.*</title>'"
<title>Cats vs Dogs!</title>
<title>Cats vs Dogs -- Result</title>

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$

```

Figure 36 — Stack fournie déployée sur Swarm : accessible seulement depuis le cluster (ports non relayés vers l'hôte).

Adaptation : suppression des ports, ajout du réseau `web` et des labels Traefik sur `vote` et `result` :

```

MINGW32: c:/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ git diff --no-index stacks/voting.original.yml stacks/voting.yml | grep -E "^[+-]" | head -n 30
warning: in the working copy of 'stacks/voting.original.yml', CRLF will be replaced by LF the next time Git touches it
- ports:
-   - 8080:80
+   - web
+ labels:
+   - "traefik.enable=true"
+   - "traefik.http.routers.vote.rule=Host(`vote.swarm.localhost`)"
+   - "traefik.http.routers.vote.entrypoints=web"
+   - "traefik.http.services.vote.loadbalancer.server.port=80"
- ports:
-   - 8081:80
+   - web
+ deploy:
+   labels:
+   - "traefik.enable=true"
+   - "traefik.http.routers.result.rule=Host(`result.swarm.localhost`)"
+   - "traefik.http.routers.result.entrypoints=web"
+   - "traefik.http.services.result.loadbalancer.server.port=80"
+ web:
+   external: true

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker cp stacks/voting.yml swarm-manager-1:/home/manager/

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec -w /home/manager swarm-manager-1 docker stack deploy -c voting.yml voting
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service voting_db (id: do9517lso4ufakte822vy6dio)
Updating service voting_vote (id: ydyex57lyu9oza2wn1f5cqbrp)
Updating service voting_result (id: s2ina04v28t44uey5jvah8p55)
Updating service voting_worker (id: fchpruwvd08k96ugbbzt1hxt8)
Updating service voting_redis (id: nniflpk70jttac65x91gbgtvp)

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$

```

Figure 37 — Différences avec la stack d'origine, puis mise à jour de la stack.

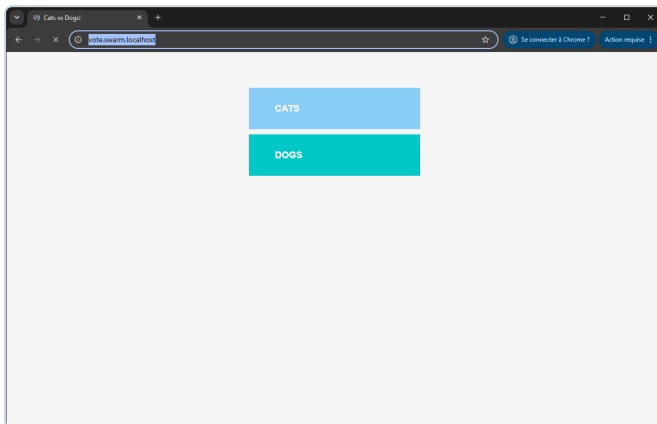


Figure 38 — http://vote.swarm.localhost

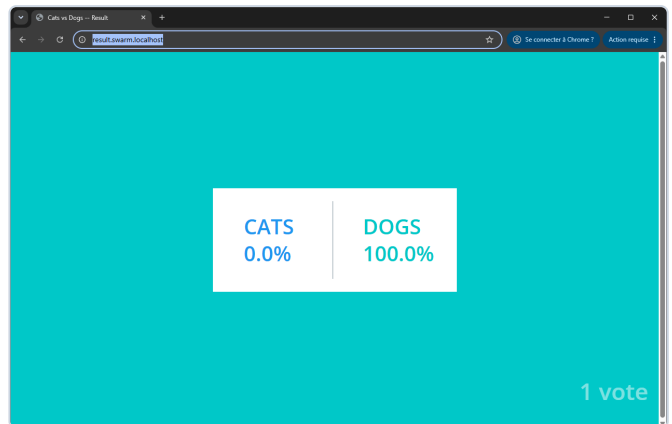


Figure 39 — http://result.swarm.localhost

Ex. 3 — Portainer

Stack officielle pour Swarm : un **agent** en mode global (un par nœud, il remonte l'état de sa machine) et le **serveur** sur le manager. Adaptation : ports retirés, publication via Traefik sur le port 9000.

stacks/portainer.yml (extrait)

```
portainer:
  image: portainer/portainer-ce:latest
  command: -H tcp://tasks.agent:9001 --tlsskipverify
  volumes:
    - portainer_data:/data
  networks:
    - agent_network
    - web
  deploy:
    mode: replicated
    replicas: 1
    placement:
      constraints: [node.role == manager]
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.portainer.rule=Host(`portainer.swarm.localhost`)"
    - "traefik.http.routers.portainer.entrypoints=web"
    - "traefik.http.services.portainer.loadbalancer.server.port=9000"
```

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 docker service logs portainer.portainer.2>&1 | grep -A2 setup_token
portainer.portainer.1.0p7ou7kewfxa@manager | setup_token=2229dbea53803e8cc495fec135c98ec143cd25f7065f9546f3abeb2a50dd0b4c
portainer.portainer.1.0p7ou7kewfxa@manager |
portainer.portainer.1.0p7ou7kewfxa@manager | Paste it into the setup screen, or send it in the X-Setup-Token header.

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$
```

Figure 40 — La version actuelle exige un setup token, lu dans les logs du service.

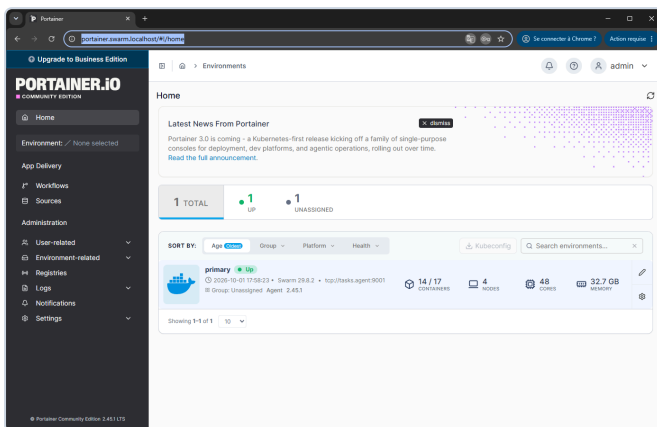


Figure 41 — Environnement Swarm détecté : 4 nœuds.

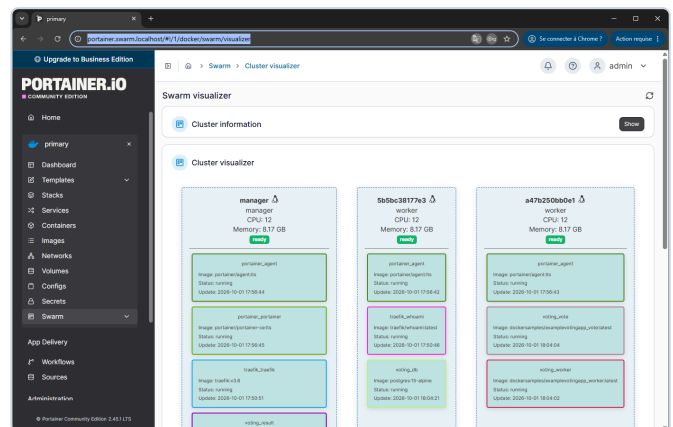


Figure 42 — Visualiseur : répartition des tâches par nœud.

Ex. 4 — Redéployer l'app de vote depuis Portainer

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 docker stack ls
NAME                SERVICES
portainer            2
traefik              2
voting              5

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ docker exec swarm-manager-1 docker stack rm voting
Removing service voting_db
Removing service voting_redis
Removing service voting_result
Removing service voting_vote
Removing service voting_worker
Removing network voting_backend
Removing network voting_frontend

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$ sleep 5; docker exec swarm-manager-1 docker stack ls
NAME                SERVICES
portainer            2
traefik              2

souha@PCSouhaill MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp4-traefik-portainer)
$
```

Figure 43 — Suppression de la stack en ligne de commande.

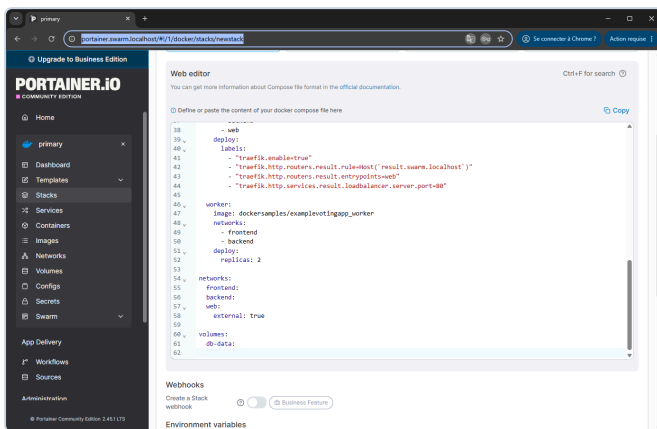


Figure 44 — Nouvelle stack « voting » dans l'éditeur web.

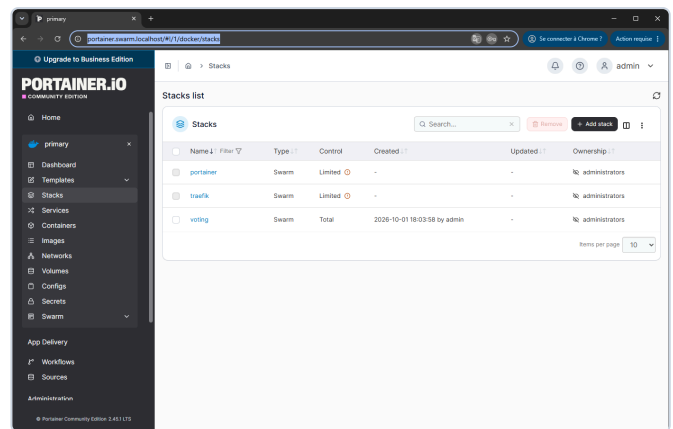


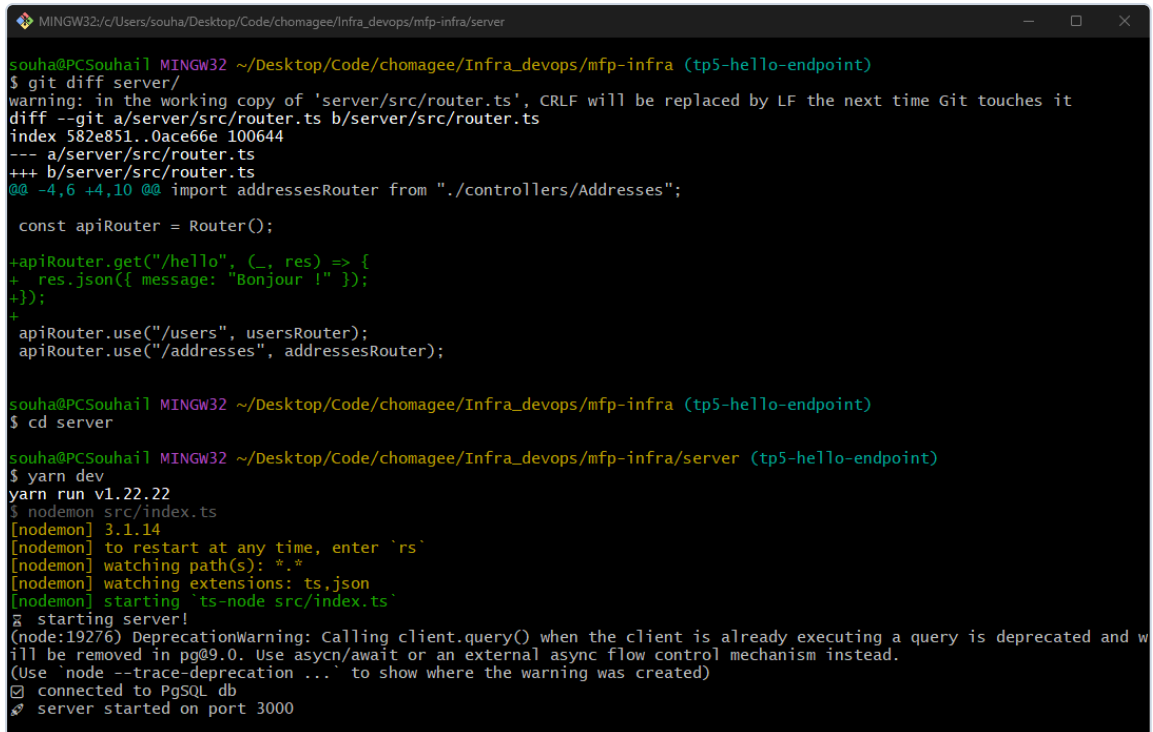
Figure 45 — voting passe en contrôle « Total ».

Les stacks créées en CLI apparaissent en contrôle **Limited** (Portainer ne connaît pas leur fichier) ; celle créée depuis Portainer est en contrôle **Total** : modifiable et redéployable depuis l'interface.

TP 5 — Déploiement continu avec Shepherd

Le cluster tourne en local et ne peut pas recevoir de webhook depuis GitHub : on utilise l'approche **polling**. Watchtower n'étant plus maintenu ni compatible Swarm, l'outil retenu est **Shepherd**.

Ex. 1.1 — Endpoint « Bonjour ! »



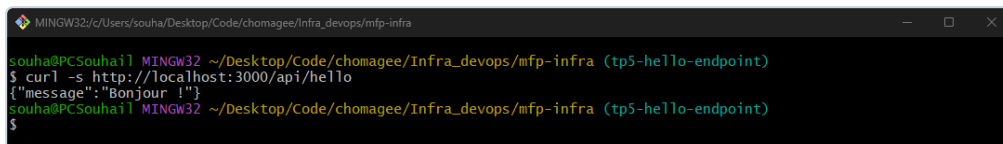
```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/server
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-hello-endpoint)
$ git diff server/
warning: in the working copy of 'server/src/router.ts', CRLF will be replaced by LF the next time Git touches it
diff --git a/server/src/router.ts b/server/src/router.ts
index 582e851..0ace66e 100644
--- a/server/src/router.ts
+++ b/server/src/router.ts
@@ -4,6 +4,10 @@ import addressesRouter from "../controllers/Addresses";

const apiRouter = Router();
+apiRouter.get("/hello", (_, res) => {
+  res.json({ message: "Bonjour !" });
+});
+
apiRouter.use("/users", usersRouter);
apiRouter.use("/addresses", addressesRouter);

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-hello-endpoint)
$ cd server

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/server (tp5-hello-endpoint)
$ yarn dev
yarn run v1.22.22
$ nodemon src/index.ts
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): "."
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/index.ts`
starting server!
(node:19276) DeprecationWarning: Calling client.query() when the client is already executing a query is deprecated and will be removed in pg@9.0. Use async/await or an external async flow control mechanism instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
connected to PostgreSQL db
server started on port 3000
```

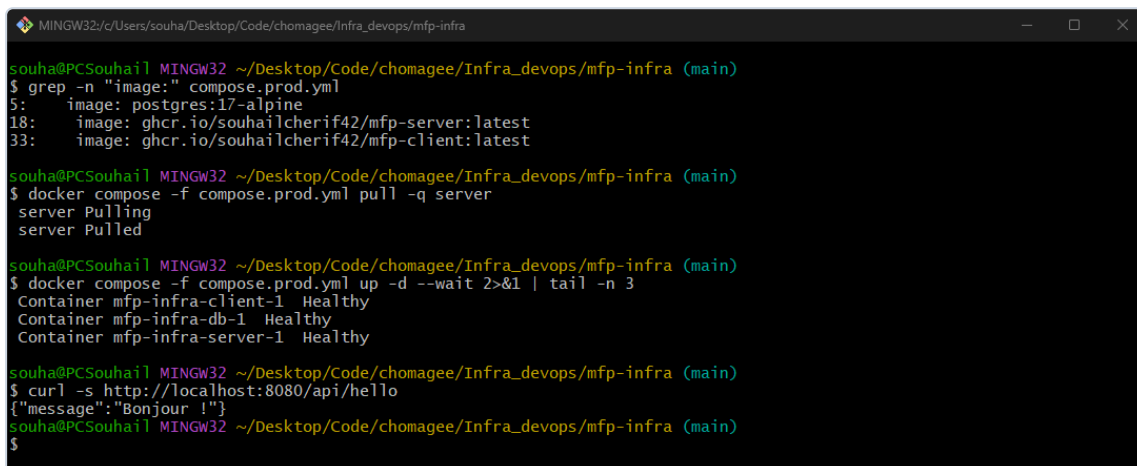
Figure 46 — Nouvel endpoint GET /api/hello, testé en développement.



```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-hello-endpoint)
$ curl -s http://localhost:3000/api/hello
{"message": "Bonjour !"}
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-hello-endpoint)
$
```

Figure 47 — Réponse en local.

Après merge de la PR, la CI publie la nouvelle image ; elle est testée avec `compose.prod.yml`, sans aucun build :



```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ grep -n "image:" compose.prod.yml
5: image: postgres:17-alpine
18: image: ghcr.io/souhailcherif42/mfp-server:latest
33: image: ghcr.io/souhailcherif42/mfp-client:latest

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose -f compose.prod.yml pull -q server
server Pulling
server Pulled

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker compose -f compose.prod.yml up -d --wait 2>&1 | tail -n 3
Container mfp-infra-client-1 Healthy
Container mfp-infra-db-1 Healthy
Container mfp-infra-server-1 Healthy

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ curl -s http://localhost:8080/api/hello
{"message": "Bonjour !"}
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$
```

Figure 48 — Image publiée testée localement avec Docker Compose.

Ex. 1.2 et 1.3 — Stack MFP créée depuis Portainer

Tag générique `main` (pas le SHA) ; deux réseaux : `internal` pour API ↔ BDD, `web` pour Traefik.

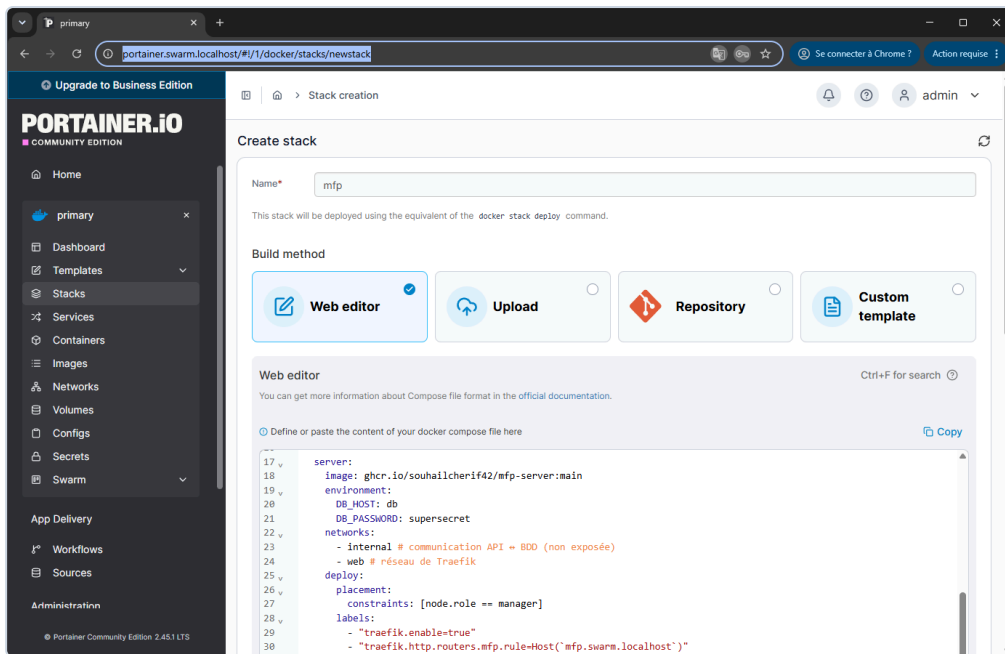


Figure 49 — Création de la stack mfp dans Portainer.

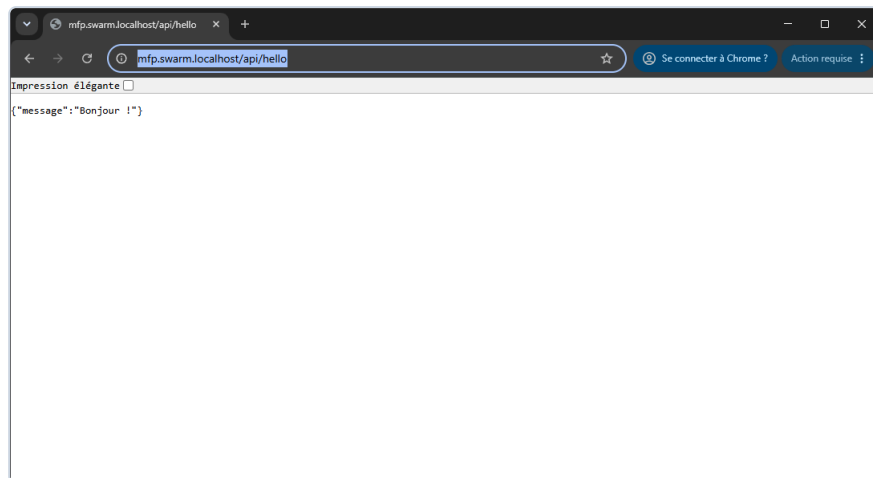


Figure 50 — API accessible via Traefik : `http://mfp.swarm.localhost/api/hello`.

Ex. 2.1 et 2.2 — Shepherd et ses logs

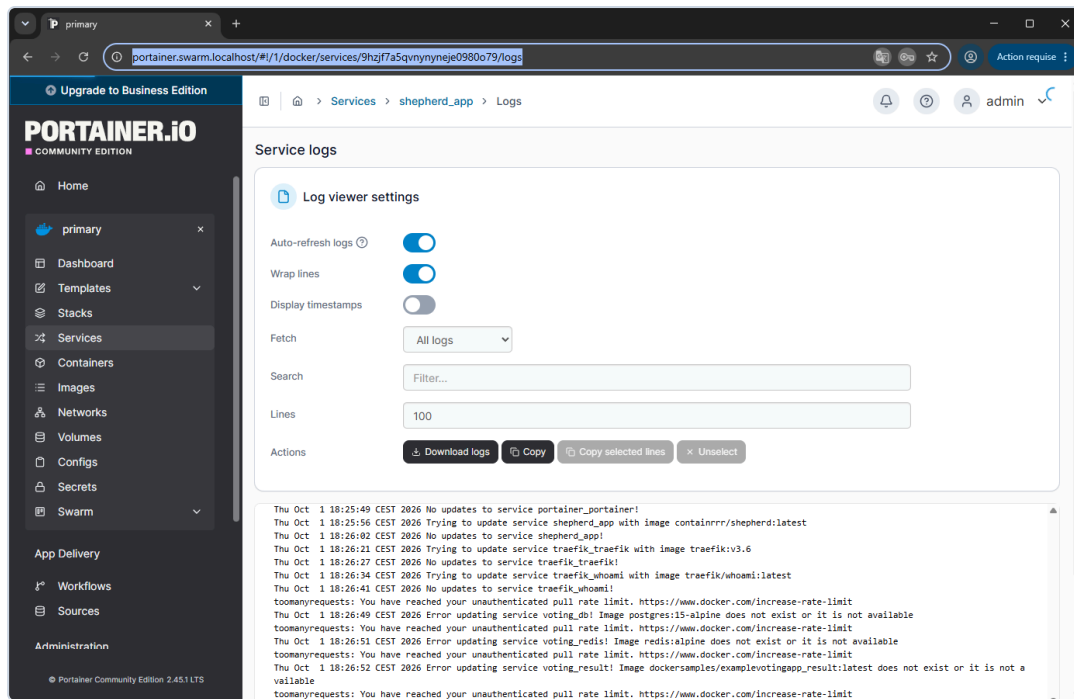


Figure 51 — Logs de Shepherd dans Portainer.

Ce que montrent les logs : toutes les minutes (`SLEEP_TIME=1m`), Shepherd parcourt *tous* les services du cluster et compare l'image en service au registre (« No updates to service ... »). Effet de bord réel : ces vérifications ont épuisé le quota de pulls anonymes de Docker Hub — `toomanyrequests` — pour les images hébergées sur Docker Hub. Notre image sur GHCR n'est pas concernée.

Ex. 2.3 — « Hello ! » sans intervention

Le message passe à « Hello ! » ; la PR est mergée à **18:30:17**. Aucune autre action :

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker exec swarm-manager-1 docker service logs --raw shepherd_app 2>&1 | grep mfp_server | tail -n 4
Thu Oct 1 18:32:11 CEST 2026 Trying to update service mfp_server with image ghcr.io/souhailcherif42/mfp-server:main
Thu Oct 1 18:32:17 CEST 2026 No updates to service mfp_server!
Thu Oct 1 18:34:14 CEST 2026 Trying to update service mfp_server with image ghcr.io/souhailcherif42/mfp-server:main
Thu Oct 1 18:34:36 CEST 2026 Service mfp_server was updated!

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ docker exec swarm-manager-1 docker service ps mfp_server --format "table {{.Name}}\t{{.Image}}\t{{.DesiredState}}\t{{.CurrentState}}\n"
table {{.Name}}\t{{.Image}}\t{{.DesiredState}}\t{{.CurrentState}}\n
NAME\tIMAGE\tDESIRED STATE\tCURRENT STATE\n
mfp_server.1\tghcr.io/souhailcherif42/mfp-server:main\tRunning\tRunning 33 seconds ago\n
  \ mfp_server.1\tghcr.io/souhailcherif42/mfp-server:main\tShutdown\tShutdown 34 seconds ago\n
  \ mfp_server.1\tghcr.io/souhailcherif42/mfp-server:main\tShutdown\tComplete 16 minutes ago\n

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$ curl -s http://mfp.swarm.localhost/api/hello
{"message":"Hello !"}
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (main)
$
```

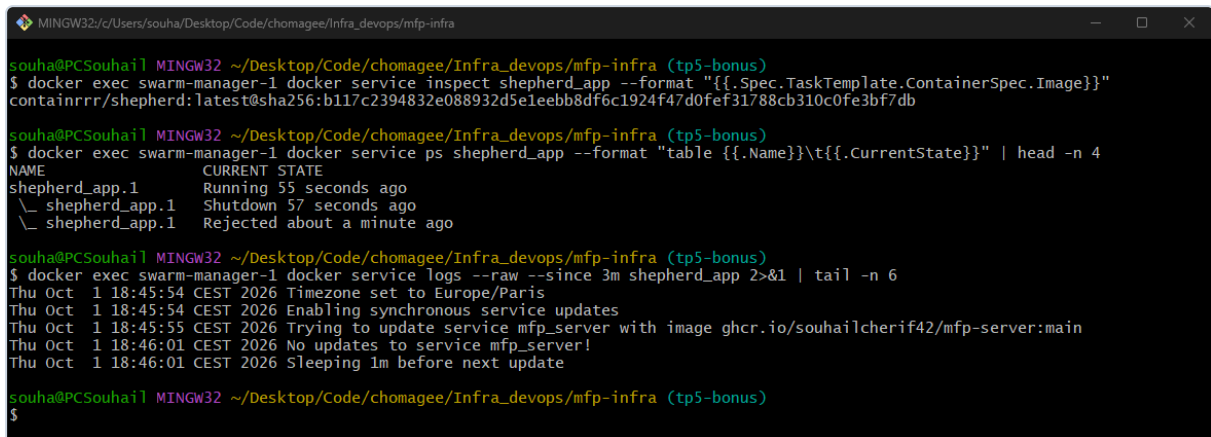
Figure 52 — 18:32 : rien de nouveau (la CI tourne encore) ; 18:34:36 : service mis à jour, l'API répond « Hello ! ».

Résultat attendu, en ~4 minutes : le temps de la CI (tests + build + push) puis au plus une minute de polling. Pour diagnostiquer un blocage : l'onglet Actions de GitHub (l'image a-t-elle été publiée ?), puis les logs de Shepherd et `docker service ps mfp_server` (la mise à jour a-t-elle été tentée, a-t-elle échoué ?).

Bonus 1 et 2 — Filtrage et nettoyage

stacks/shepherd.yml

```
# Shepherd (https://github.com/containrrr/shepherd) : CD par polling pour Docker Swarm.
# Toutes les SLEEP_TIME, il vérifie si l'image de chaque service a changé sur le registre
# (même tag, nouveau digest) et lance "docker service update" si c'est le cas.
services:
  app:
    # image épinglée par digest : pas de requête Docker Hub au redémarrage (quota anonyme limité)
    image:
      containrrr/shepherd:latest@sha256:b117c2394832e088932d5e1eebb8df6c1924f47d0fef31788cb310c0fe3bf7db
    environment:
      TZ: 'Europe/Paris'
      SLEEP_TIME: '1m'
      # Seuls les services qui le demandent explicitement sont mis à jour
      FILTER_SERVICES: 'label=autoupdate_enabled=true'
      # Après une mise à jour, ne garde que la dernière image du service (sur le nœud de Shepherd)
      IMAGE_AUTOCLEAN_LIMIT: '1'
      VERBOSE: 'true'
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
  deploy:
    placement:
      constraints:
        - node.role == manager
```



```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service inspect shepherd_app --format "{{.Spec.TaskTemplate.ContainerSpec.Image}}"
containrrr/shepherd:latest@sha256:b117c2394832e088932d5e1eebb8df6c1924f47d0fef31788cb310c0fe3bf7db

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service ps shepherd_app --format "table {{.Name}}\t{{.CurrentState}}" | head -n 4
NAME                CURRENT STATE
shepherd_app.1      Running 55 seconds ago
  \ shepherd_app.1  Shutdown 57 seconds ago
  \ shepherd_app.1  Rejected about a minute ago

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service logs --raw --since 3m shepherd_app 2>&1 | tail -n 6
Thu Oct 1 18:45:54 CEST 2026 Timezone set to Europe/Paris
Thu Oct 1 18:45:54 CEST 2026 Enabling synchronous service updates
Thu Oct 1 18:45:55 CEST 2026 Trying to update service mfp_server with image ghcr.io/souhaïlcherif42/mfp-server:main
Thu Oct 1 18:46:01 CEST 2026 No updates to service mfp_server!
Thu Oct 1 18:46:01 CEST 2026 Sleeping 1m before next update

souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$
```

Figure 53 — Avec FILTER_SERVICES, Shepherd ne vérifie plus que mfp_server.

Incident réel. La mise à jour de la stack Shepherd a été refusée en boucle : Swarm revérifie l'image sur Docker Hub, qui répondait `429 Too Many Requests`. La mise à jour de la stack MFP depuis Portainer a aussi recréé la tâche PostgreSQL, restée bloquée 17 minutes pour la même raison ; pendant ce temps une mise à jour de l'API a échoué (« ENOTFOUND db »). **Correctif :** images Docker Hub épinglées par digest (`image@sha256:...`) : un nœud qui possède déjà l'image ne contacte plus le registre.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service ps shepherd_app --no-trunc --format "{{.CurrentState}} | {{.Error}}" | head -n 2 | cu
t -c 1-170
Rejected less than a second ago | "unknown: failed to resolve reference "docker.io/containerrr/shepherd:latest": unexpected status
from HEAD request to https://registry-1.
Rejected 5 seconds ago | "unknown: failed to resolve reference "docker.io/containerrr/shepherd:latest": unexpected status from HEAD
request to https://registry-1.docker.io

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker images containerrr/shepherd
IMAGE ID DISK USAGE C
ONTENT SIZE EXTRA
containerrr/shepherd:latest@sha256:b117c2394832e088932d5e1eebb8df6c1924f47d0fef31788cb310c0fe3bf7db b117c2394832 563MB
147MB

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker cp stacks/shepherd.yml swarm-manager-1:/home/manager/

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec -w /home/manager swarm-manager-1 docker stack deploy --resolve-image never -c shepherd.yml shepherd
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service shepherd_app (id: 9hzjf7a5qvnynynje0980o79)

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$
```

Figure 54 — Diagnostic : tâches rejetées (HTTP 429) alors que l'image est présente sur le manager.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service logs --raw --since 25m shepherd_app 2>&1 | grep -A12 "19:01:57 CEST 2026 Service mfp_
server was updated"
Thu Oct 1 19:01:57 CEST 2026 Service mfp_server was updated!
Thu Oct 1 19:01:57 CEST 2026 Cleaning up old docker images, leaving last 1
Deleted Containers:
b7c55a14d2e89fd1db5c4b656c4cc95b6ebaf7a1a1b4ed27bce840661051e009
9cbf1f8041c55098a4e25379dc65274eead30b953ebaca95c42c684194ee6aa6
485bf566e4a34b6cbacf8e585241099f19528cb1e47d70b1a3bdd3ebf9fa510a
40e3d059a34bfa84bcd161d8fade58517639fbb68090862cc301df87f7ae4cb
deddcccfd7377639d3288414c58f2258913516e2f992a60f38a727fe6e0787b0

Total reclaimed space: 53.25kB
Untagged: ghcr.io/souhailcherif42/mfp-server:main@sha256:c4482809ac64460afab0d6b84f23ad420a0588198f15929ee566204f11dfb2c1
Deleted: sha256:c4482809ac64460afab0d6b84f23ad420a0588198f15929ee566204f11dfb2c1
Thu Oct 1 19:01:58 CEST 2026 Sleeping 1m before next update

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker images ghcr.io/souhailcherif42/mfp-server --format "table {{.ID}}\t{{.CreatedSince}}"
IMAGE ID CREATED
b5f00864908b 9 minutes ago
c43ac3f04982 37 minutes ago
d206d0feb9a8 53 minutes ago

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$
```

Figure 55 — Nettoyage après mise à jour : l'image précédente est supprimée.

Limite constatée du nettoyage : il ne supprime que l'image précédente, et seulement sur le nœud où tourne Shepherd (le manager). Les images plus anciennes, référencées uniquement par digest, ne sont pas reconnues par son filtre.

Bonus 3 — MFP sur un autre nœud

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker node update --label-add nodeId=1 manager
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker node update --label-add nodeId=2 $(docker exec swarm-node-1 hostname)
5b5bc38177e3
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker node update --label-add nodeId=3 $(docker exec swarm-node-2 hostname)
c9a838dc7311
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker node update --label-add nodeId=4 $(docker exec swarm-node-3 hostname)
a47b250bb0e1
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 sh -c 'docker node inspect $(docker node ls -q) --format "{{.Description.Hostname}} role={{.Spec.Role}} labels={{.Spec.Labels}}"'
5b5bc38177e3 role=worker labels=map[nodeId:2]
a47b250bb0e1 role=worker labels=map[nodeId:4]
c9a838dc7311 role=worker labels=map[nodeId:3]
manager role=manager labels=map[nodeId:1]
souha@PCSouhaïl MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$
```

Figure 56 — Labels nodeId posés sur les 4 nœuds depuis le manager.

stacks/mfp.yml (version finale, extrait)

```
# API MFP + sa base PostgreSQL, déployées depuis Portainer.
# Tag générique "main" : chaque push sur main produit une nouvelle image sous ce même tag,
# ce qui permet à Shepherd de détecter les mises à jour.
services:
  db:
    # épinglée par digest : le nœud utilise l'image locale sans interroger Docker Hub
    image: postgres:17-alpine@sha256:b0f9560a2de083e2cc7382e75f808c7381a32852a7ec49117deedb300e552b24
    environment:
      POSTGRES_PASSWORD: supersecret
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - internal
    deploy:
      placement:
        constraints: [node.labels.nodeId == 2] # nœud worker étiqueté nodeId=2

  server:
    image: ghcr.io/souhailcherif42/mfp-server:main
    environment:
      DB_HOST: db
      DB_PASSWORD: supersecret
    networks:
      - internal # communication API ↔ BDD (non exposée)
      - web # réseau de Traefik
    deploy:
      placement:
        constraints: [node.labels.nodeId == 2] # nœud worker étiqueté nodeId=2
    update_config:
      order: start-first # le nouveau conteneur démarre avant l'arrêt de l'ancien
      failure_action: rollback # retour à l'image précédente si la MAJ échoue
    labels:
      - "autoupdate_enabled=true" # Shepherd met à jour ce service (FILTER_SERVICES)
      - "traefik.enable=true"
      - "traefik.http.routers.mfp.rule=Host(`mfp.swarm.localhost`)"
      - "traefik.http.routers.mfp.entrypoints=web"
      - "traefik.http.services.mfp.loadbalancer.server.port=3000"
```

J'ai aussi ajouté un `update_config` : *start-first* (le nouveau conteneur démarre avant l'arrêt de l'ancien) et *rollback* en cas d'échec — deux leçons directes de l'incident précédent.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker stack ps mfp --filter desired-state=running --format "table {{.Name}}\t{{.Node}}\t{{.CurrentState}}\n"
NAME                NODE                CURRENT STATE
mfp_db.1             5b5bc38177e3       Running 19 minutes ago
mfp_server.1         5b5bc38177e3       Running 16 minutes ago

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ cd bruno && bru run --env swarm | tail -n 22
✓ address geocoded
5-list-addresses (200 OK) - 28 ms
Tests
✓ one address
6-hello (200 OK) - 7 ms
Tests
✓ hello message

Execution Summary


| Metric        | Result       |
|---------------|--------------|
| Status        | ✓ PASS       |
| Requests      | 6 (6 Passed) |
| Tests         | 6/6          |
| Assertions    | 0/0          |
| Duration (ms) | 1867         |



souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra/bruno (tp5-bonus)
$
```

Figure 57 — API et BDD sur le worker nodeId=2 ; tous les tests Bruno passent à travers Traefik.

```
MINGW32/c/Users/souha/Desktop/Code/chomagee/Infra_devops/mfp-infra
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service logs --raw --since 15m shepherd_app 2>&1 | grep mfp_server | tail -n 3
Thu Oct 1 19:14:28 CEST 2026 Service mfp_server was updated!
Thu Oct 1 19:15:30 CEST 2026 Trying to update service mfp_server with image ghcr.io/souhailcherif42/mfp-server:main
Thu Oct 1 19:15:36 CEST 2026 No updates to service mfp_server!

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ docker exec swarm-manager-1 docker service ps mfp_server --format "table {{.Name}}\t{{.Node}}\t{{.DesiredState}}\t{{.CurrentState}}\n" | head -n 3
NAME                NODE                DESIRED STATE  CURRENT STATE
mfp_server.1         5b5bc38177e3       Running        Running about a minute ago
\ mfp_server.1       5b5bc38177e3       Shutdown       Shutdown about a minute ago

souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$ curl -s http://mfp.swarm.localhost/api/hello
{"message":"Hello !"}
souha@PCSouhail MINGW32 ~/Desktop/Code/chomagee/Infra_devops/mfp-infra (tp5-bonus)
$
```

Figure 58 — La CD fonctionne toujours : nouvelle image déployée automatiquement sur le worker.

Bilan

TP	Réalisé	Non réalisé
Dockerisation	Tous les exercices et « pour aller plus loin » (-140 Mo)	—
CI	Ex. 2 et 3, filtre par dossier, Bruno dans la CI	Test E2E Cypress
Swarm	Ex. 1 à 3	—
Ansible	Ex. 4 et 5 ; correction du playbook ; version idempotente	Bonus VirtualBox
Traefik & Portainer	Ex. 1 à 4	—
CD (Shepherd)	Ex. 1, 2 et les trois bonus	—

Au-delà des outils, ces TP montrent ce qui rend une chaîne DevOps fiable : des contrôles automatiques qui bloquent réellement (la CI a refusé un build cassé et une PR erronée), des procédures rejouables sans effet de bord (playbook idempotent) et une observation attentive des effets indirects. L'incident du quota Docker Hub illustre ce dernier point : un outil de déploiement continu mal filtré a fini par empêcher le redémarrage d'une base de données. Il a conduit à trois améliorations concrètes : filtrage des services, images épinglées par digest, et mises à jour *start-first* avec rollback.